Chapter 6 Auto Regressive models and Transformers

1. Understand what is an **auto-regressive** model
2. Describe bow **RNN** work
3. Undestand the concept of **LSTM** and its **limitations**
4. Describe the **basic ideas** behind a **Transformer** architecture
5. **Byte-level BPE** tokenization (learn merges from raw UTF-8 bytes)
6. **Decoder-only Transformer** with **causal self-attention**
7. A small **train/evaluate/generate** loop on a simple corpus (Tiny Shakespeare)
8. A **worked attention example** showing the actual $QK^\top$ scores, causal masking, and the softmax weights

License

Autoregressive Models

An **autoregressive (AR)** model defines a probability distribution over a sequence by factorizing it into a product of **next-step conditionals**.

Let a sequence be $x_{1:T} = (x_1, \ldots, x_T)$. The autoregressive factorization is:

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{1:t-1}).$$

For language modeling with tokens $s_t \in \{1, \ldots, V\}$:

$$p(s_{1:T}) = \prod_{t=1}^{T} p(s_t \mid s_{<t}), \quad p(s_t \mid s_{<t}) = \text{Categorical}(\pi_t), \quad \pi_t = \text{softmax}(z_t).$$

$s_t \sim \text{Categorical}(\pi_t)$

means:

$\Pr(s_t = i \mid s_{<t}) = \pi_{t,i} \quad \text{for } i = 1, \ldots, V.$

where V is the vocabulary (all tokens).

So

$p(s_t \mid s_{<t}) = \text{Categorical}(\pi_t)$

is shorthand for "the conditional distribution of the next token is categorical with parameter pi_t." So the $Categorical$ may give us the next probably word, for example.

Two distinct stages always exist:

- **Training (teacher forcing):** you condition on the *ground-truth* prefix $x_{<t}$ to predict x_t.
- **Sampling (generation):** you condition on the *model's own previously generated* samples.

Simple Autoregressive Example: Predicting the Next Digit (0/1) in a Pattern

We build a tiny **autoregressive** (AR) model over binary digits.

Let the sequence be:

$$x_{1:T}, \quad x_t \in \{0, 1\}.$$

The **autoregressive factorization** is:

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{1:t-1}).$$

A patterned dataset

Consider a deterministic pattern that alternates:

$$\underline{0, 1, 0, 1, 0, 1, \ldots}$$

So the rule is:

- if the last digit is 0, the next is 1
- if the last digit is 1, the next is 0

Formally, for $t \geq 2$:

$$x_t = 1 - x_{t-1}.$$

A simple autoregressive model for next-digit prediction

Assume our model only looks at the **most recent digit** (a 1-step Markov AR model):

$$p_\theta(x_t \mid x_{1:t-1}) = p_\theta(x_t \mid x_{t-1}).$$

We represent the probability of $x_t = 1$ using a sigmoid:

$$p_\theta(x_t = 1 \mid x_{t-1}) = \sigma(w\, x_{t-1} + b),$$

and therefore:

$$p_\theta(x_t = 0 \mid x_{t-1}) = 1 - \sigma(w\, x_{t-1} + b).$$

Define:

$$\hat{p}_t := p_\theta(x_t = 1 \mid x_{t-1}) = \sigma(w x_{t-1} + b).$$

A Bernoulli distribution with parameter p has:

- $P(X = 1) = p$
- $P(X = 0) = 1 - p$

We want **one formula** that works for both outcomes $x \in \{0, 1\}$.

That standard Bernoulli PMF is:

$$P(X = x) = p^x (1 - p)^{1 - x}.$$

Therefore, the likelihood for the observed binary target x_t is:

$$p_\theta(x_t \mid x_{t-1}) = \hat{p}_t^{\,x_t} (1 - \hat{p}_t)^{(1 - x_t)}.$$

Training stage (teacher forcing)

Given a training sequence (e.g., $0, 1, 0, 1, 0, 1$), we minimize the negative log-likelihood:

$$\mathcal{L}(\theta) = -\sum_{t=2}^{T} \log p_\theta(x_t \mid x_{t-1}).$$

This becomes the standard binary cross-entropy:

$$\mathcal{L}(\theta) = -\sum_{t=2}^{T} \Big[x_t \log(\hat{p}_t) + (1 - x_t) \log(1 - \hat{p}_t) \Big].$$

Teacher forcing means: when predicting x_t, we condition on the **true** previous digit x_{t-1} from the dataset.

Sampling stage (generation)

To generate a new sequence, choose a starting digit x_1 (seed), then repeatedly sample:

1. Compute

$$\hat{p}_t = \sigma(w x_{t-1} + b)$$

2. Sample

$$x_t \sim \text{Bernoulli}(\hat{p}_t)$$

3. Append x_t and continue

For example, if the model learned the alternating pattern well:

- start with $x_1 = 0$
- it produces $x_2 \approx 1, x_3 \approx 0, x_4 \approx 1, \ldots$

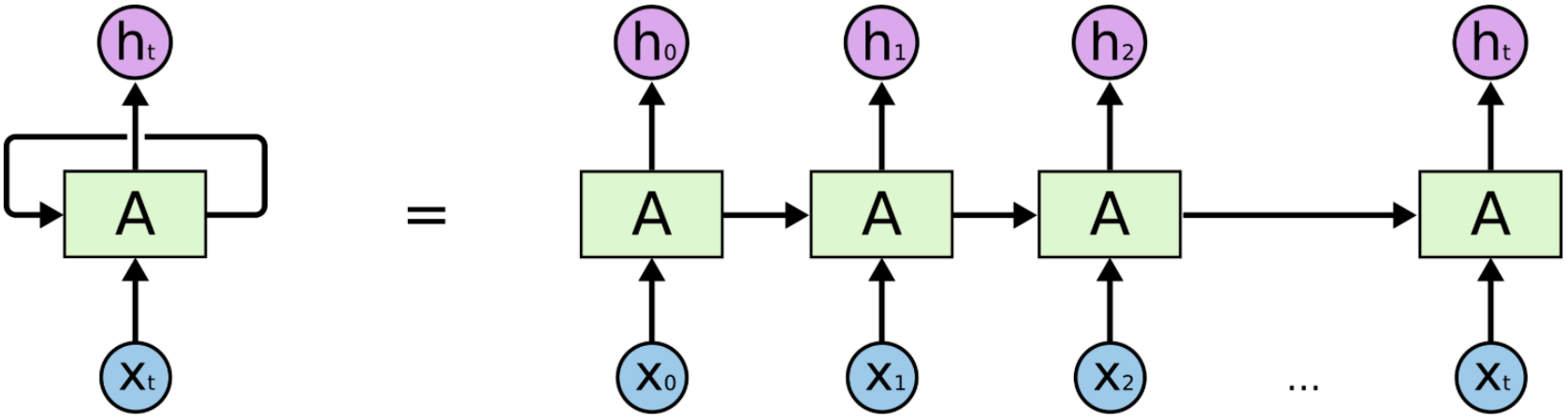
What makes this “autoregressive”?

At generation time, the model’s prediction of x_t is conditioned on **previous generated digits**:

$$x_{t-1} \to \hat{p}_t \to x_t,$$

so the sequence is produced **one step at a time**, always using the past as context. In this example, the context size is **one** because we only look at the digit that came before. In general, we could look at a larger **context window** but this is not necessary in this case.

Recurrent Neural Networks (RNNs)



Source: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Key idea

Maintain a **hidden state** h_t that summarizes the past. Each step updates the state and predicts the next token. Here A is a neural network which is trained as **each** token X_i that passes through it in a sequence. The important thing to remeber is that there is only one neural network A whose **weights** are changed in response to each X_i it sees in a sequenece.

What is learned (parameters)

Assume:

- Vocabulary size: V indicating the words (and eventually tokens) in a corpose (e.g., "hello", "how", "are", ...)
- Embedding dimension: d representing the dimension of the word embeddings
- Hidden size: H representing a state that $remembers$ the "context" of previously seen words

The learnable parameters are:

- **Embedding matrix** $E \in \mathbb{R}^{V \times d}$

    - Looks up a dense vector for each token.
    - **Learns semantic/syntactic token representations** (e.g., similar tokens end up with similar embeddings).
- **Input-to-hidden weights** $W_{xh} \in \mathbb{R}^{H \times d}$
    - Converts the current input embedding x into a contribution to the new hidden state.
    - **Learns how the current token should influence memory**.
- **Hidden-to-hidden (recurrent) weights** $W_{hh} \in \mathbb{R}^{H \times H}$
    - Transforms the previous hidden state h_{t-1} into a contribution to the new hidden state h_t.
    - **Learns how to carry and update temporal memory** (how past information persists/decays/combines).
- **Hidden bias** $b_h \in \mathbb{R}^H$
    - Shifts the pre-activation for the hidden state.
    - **Learns baseline offsets** for hidden activations.
- **Hidden-to-output weights** $W_{hy} \in \mathbb{R}^{V \times H}$
    - Maps hidden state to logits (or output) over the vocabulary. So each v in V gets assigned a number (or probability).
    - **Learns how internal features correspond to next-token predictions**.
- **Output bias** $b_y \in \mathbb{R}^V$
    - Shifts logits for each token.
    - **Learns baseline token preferences** (e.g., frequency effects).

Collectively, the parameter set is:

$$\theta = \{E, W_{xh}, W_{hh}, b_h, W_{hy}, b_y\}.$$

Core equations

Let the input token be $x_t \in \{1, \dots, V\}$ and the embedding lookup be:

$$e(x_t) = E[x_t] \in \mathbb{R}^d.$$

Hidden state update:

$$h_t = \phi\left(W_{xh} e(x_t) + W_{hh} h_{t-1} + b_h\right),$$

where ϕ is typically $\tanh$ or ReLU.

Output logits and probabilities:

$$z_{t+1} = W_{hy} h_t + b_y, \quad p(x_{t+1} \mid x_{\le t}) = \mathrm{softmax}(z_{t+1}).$$

Training stage (teacher forcing)

Given a training sequence $x_{1:T}$, minimize negative log-likelihood (cross-entropy):

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

What happens during training:

- At each step, the model consumes the **true** token x_t (via its embedding) to compute h_t.
- It then predicts the distribution for the next token (or equivalently predicts x_t from h_{t-1} depending on indexing).
- Gradients are computed by **backpropagation through time (BPTT)**, updating:
  - E to create useful token vectors,
  - W_{xh}, W_{hh}, b_h to update memory,
  - W_{hy}, b_y to map hidden features to next-token probabilities.

Sampling stage (generation)

Start with a seed token (e.g., $x_1 = \langle \mathrm{BOS} \rangle$ or a prompt) and an initial hidden state h_0 (often zeros).

For $t = 1, 2, \ldots$:

1. Embed current token: $e(x_t) = E[x_t]$
2. Update hidden state:

$$h_t = \phi\left(W_{xh} e(x_t) + W_{hh} h_{t-1} + b_h\right)$$

3. Compute next-token distribution:

$$p(x_{t+1} \mid x_{\leq t}) = \mathrm{softmax}(W_{hy} h_t + b_y)$$

4. Sample:

$$x_{t+1} \sim \mathrm{Categorical}\left(p(\cdot)\right)$$

   (or take $\arg\max$)
5. Repeat until $\langle \mathrm{EOS} \rangle$ or a length limit

Key difference from training: during sampling, the model conditions on its **own generated tokens**, not ground-truth tokens.

A Tiny RNN Autoregressive Model on Binary Strings (0/1)

We build an autoregressive RNN that predicts the next digit in a binary sequence:

$$x_{1:T}, \quad x_t \in \{0, 1\}.$$

We will keep **everything small**:

- Vocabulary size: $V = 2$ (tokens: 0 and 1)
- Embedding size: $d = 2$
- Hidden size: $H = 2$

Model definition (parameters and shapes)

Embedding matrix

We use a learnable embedding matrix:

$$E \in \mathbb{R}^{V \times d} = \mathbb{R}^{2 \times 2}.$$

So:

$$e(0) = E[0] \in \mathbb{R}^2, \quad e(1) = E[1] \in \mathbb{R}^2.$$

To keep the example concrete, assume simple embeddings:

$$E = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad e(0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad e(1) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

RNN parameters

Hidden update:

$$h_t = \tanh(W_{xh} e(x_t) + W_{hh} h_{t-1} + b_h)$$

with:

- $$W_{xh} \in \mathbb{R}^{H \times d} = \mathbb{R}^{2 \times 2}$$
- $$W_{hh} \in \mathbb{R}^{H \times H} = \mathbb{R}^{2 \times 2}$$
- $$b_h \in \mathbb{R}^{H} = \mathbb{R}^{2}$$

Output logits:

$$z_{t+1} = W_{hy} h_t + b_y$$

with:

- $$W_{hy} \in \mathbb{R}^{V \times H} = \mathbb{R}^{2 \times 2}$$
- $$b_y \in \mathbb{R}^{V} = \mathbb{R}^{2}$$

Next-token probabilities:

$$p(x_{t+1} \mid x_{\le t}) = \mathrm{softmax}(z_{t+1}).$$

Training setup (teacher forcing)

Given a training sequence, e.g.

$$x_{1:6} = [0, 1, 0, 1, 0, 1],$$

we train the model to predict:

$$x_2 \text{ from } x_1,\ x_3 \text{ from } x_2,\ \ldots,\ x_6 \text{ from } x_5.$$

Objective:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\le t}).$$

Teacher forcing means: when predicting x_{t+1}, we feed the **true** x_t to compute h_t.

One full forward pass (concrete numbers)

We choose specific small weights to show the mechanics.

Choose weights

Let:

$$W_{xh} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

So the hidden state depends only on the current input embedding:

$$h_t = \tanh(e(x_t)).$$

Let the output layer be:

$$W_{hy} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad b_y = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Interpretation:

- If h_t is more like $[1, 0]^\top$, prefer output token 0.
- If h_t is more like $[0, 1]^\top$, prefer output token 1.

Initialize

Set:

$$h_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Step t=1 (input $x_1 = 0$, predict x_2)

Embedding:

$$e(x_1) = e(0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Hidden:

$$h_1 = \tanh(e(0)) = \begin{bmatrix} \tanh(1) \\ \tanh(0) \end{bmatrix} \approx \begin{bmatrix} 0.761 \\ 0 \end{bmatrix}.$$

Logits for next token:

$$z_2 = W_{hy} h_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0.761 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.761 \\ -0.761 \end{bmatrix}.$$

Probabilities:

$$p(x_2 \mid x_1) = \mathrm{softmax}(z_2).$$

Since $0.761 > -0.761$, the model assigns higher probability to token **0**.

Step t=2 (input $x_2 = 1$, predict x_3)

Embedding:

$$e(x_2) = e(1) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Hidden:

$$h_2 = \tanh(e(1)) = \begin{bmatrix} \tanh(0) \\ \tanh(1) \end{bmatrix} \approx \begin{bmatrix} 0 \\ 0.761 \end{bmatrix}.$$

Logits:

$$z_3 = W_{hy} h_2 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0.761 \end{bmatrix} = \begin{bmatrix} -0.761 \\ 0.761 \end{bmatrix}.$$

Now token **1** has higher probability.

What does this show?

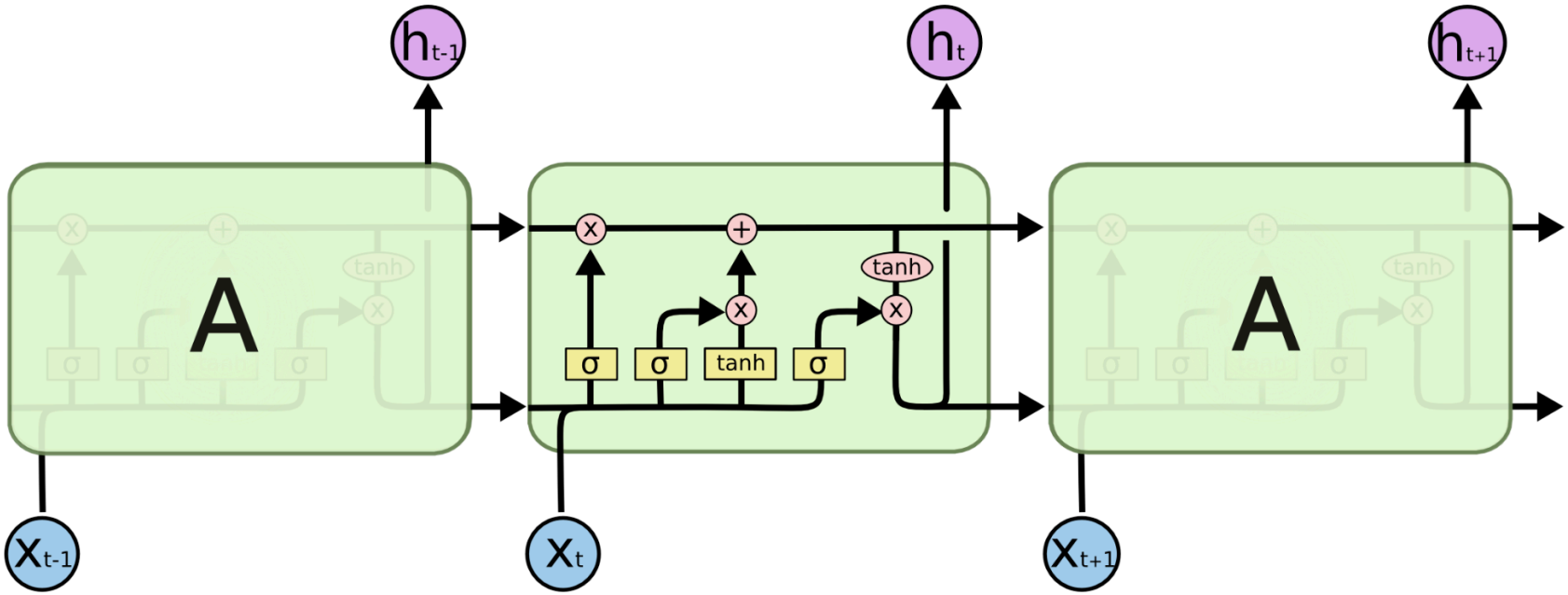
- The RNN processes tokens **sequentially**.
- Each step produces a **distribution** for the next token.
- In a real trained RNN, W_{hh} would be nonzero and learn how to store history (not just the current token).
- Training would adjust $E, W_{xh}, W_{hh}, W_{hy}$ so the next-token predictions match the dataset pattern (e.g., alternating 0/1).

Sampling stage (generation)

Given a start token x_1:

1. Compute h_1, then $p(x_2 \mid x_1)$
2. Sample x_2
3. Feed x_2 back in, compute h_2, then $p(x_3 \mid x_{1:2})$
4. Repeat to generate a full binary sequence

Long Short-Term Memory (LSTM)



Source: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

An **LSTM (Long Short-Term Memory)** is a recurrent model designed to fix a core weakness of vanilla RNNs:
it is hard for gradients and information to survive across many time steps.

The LSTM introduces a dedicated **memory cell** c_t and **gates** that learn when to:

- **forget** old information
- **write** new information
- **expose** (read out) information to make predictions

The key intuition is that the cell state c_t (top $arrow$ in the diagram above) can behave like a **nearly linear "information highway"** through time, while the gates learn *how much* to modify it.

What is learned (parameters and what each learns)

Assume:

- input embedding dimension: d
- hidden size: H

Inputs:

- $x_t \in \mathbb{R}^d$ (embedding of token x_t)
- $h_{t-1} \in \mathbb{R}^H$ (previous hidden state)
- $c_{t-1} \in \mathbb{R}^H$ (previous cell/memory)

The dimensions of h_{t-1} and c_{t-1} are typically the same.

Gate parameters (learned)

Each gate is a sigmoid unit:

$$\sigma(\cdot) \in (0, 1),$$

so it outputs a **soft on/off** control for each hidden dimension.

We have three gates and one candidate update:

- **Input gate** parameters:
  - $W_i \in \mathbb{R}^{H \times d}$ (how current input proposes writing)
  - $U_i \in \mathbb{R}^{H \times H}$ (how past hidden proposes writing)
  - $b_i \in \mathbb{R}^H$ (baseline write tendency)

  What it learns: *when and how strongly to write new information into memory.*
- **Forget gate** parameters:
  - $W_f \in \mathbb{R}^{H \times d}$
  - $U_f \in \mathbb{R}^{H \times H}$
  - $b_f \in \mathbb{R}^H$

  What it learns: *when to erase/keep each memory component.*
- **Output gate** parameters:

- $W_o \in \mathbb{R}^{H \times d}$
- $U_o \in \mathbb{R}^{H \times H}$
- $b_o \in \mathbb{R}^{H}$

What it learns: *what part of the memory should be exposed to the model's output/decision.*

- **Candidate memory (write proposal) parameters:**
  - $W_c \in \mathbb{R}^{H \times d}$
  - $U_c \in \mathbb{R}^{H \times H}$
  - $b_c \in \mathbb{R}^{H}$

  What it learns: *what new content should be written into memory (before gating).*

Output (language model) parameters (learned)

If we use the LSTM for next-token prediction over a vocabulary of size V:

- $W_{hy} \in \mathbb{R}^{V \times H}$ (map hidden features to token logits)
- $b_y \in \mathbb{R}^{V}$

What it learns: *how hidden features correspond to next-token probabilities.*

So the total learnable parameters are:

$$\theta = \{W_i, U_i, b_i,\ W_f, U_f, b_f,\ W_o, U_o, b_o,\ W_c, U_c, b_c,\ W_{hy}, b_y\}.$$

LSTM equations

Gates (sigmoid in $[0, 1]$)

Input gate

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

Forget gate

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

Output gate

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

Candidate memory content (tanh in $[-1, 1]$)

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

Cell (memory) update

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Interpretation:

- $f_t \odot c_{t-1}$ = **keep/forget** old memory
- $i_t \odot \tilde{c}_t$ = **write** new memory

Hidden state (readout from memory)

$$h_t = o_t \odot \tanh(c_t)$$

Interpretation:

- $\tanh(c_t)$ squashes memory to a bounded range
- o_t chooses **what to expose** from memory at this time

The intuition in one picture (conceptual)

Think of the cell update:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

as a learned **soft overwrite rule**:

- If $f_t \approx 1$ and $i_t \approx 0$, then $c_t \approx c_{t-1}$
  → memory is **copied forward** almost unchanged (long-term storage).
- If $f_t \approx 0$ and $i_t \approx 1$, then $c_t \approx \tilde{c}_t$
  → memory is **overwritten** with new content.
- If both are in between, LSTM performs a **learned interpolation**.

This is the mechanism that lets LSTMs keep important information for long spans and update it only when needed.

Training stage (teacher forcing for next-token prediction)

For a token sequence $x_{1:T}$, we compute hidden states sequentially:

$$(h_t, c_t) = \mathrm{LSTM}(x_t, h_{t-1}, c_{t-1}),$$

then predict the next token with:

$$z_{t+1} = W_{hy} h_t + b_y, \quad p(x_{t+1} \mid x_{\le t}) = \mathrm{softmax}(z_{t+1}).$$

Loss (negative log-likelihood):

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T-1} \log p_\theta(x_{t+1} \mid x_{\le t}).$$

Teacher forcing: at step t, you feed the **true** x_t from the dataset (not the model's sampled output).

Gradients are computed by **backpropagation through time (BPTT)**, updating all gate and output parameters.

Sampling stage (generation)

Initialize h_0, c_0 (often zeros), and start from a prompt (or a special token).

For $t = 1, 2, \ldots$:

1. Compute gates i_t, f_t, o_t and candidate $\tilde{c}_t$ from x_t, h_{t-1}

2. Update memory c_t and hidden state h_t
3. Compute next-token distribution:

$$p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(W_{hy} h_t + b_y)$$

4. Sample:

$$x_{t+1} \sim \text{Categorical}(p(\cdot))$$

5. Feed the sampled token back in and repeat

Key difference from training: during sampling, the model conditions on its **own outputs**, which can cause error accumulation (exposure bias).

Summary

The LSTM learns **when to remember, when to forget, and when to reveal memory** via gates:

- f_t = keep/forget old memory
- i_t = write new memory
- o_t = expose memory to prediction

All of this is learned through next-token prediction loss and BPTT.

Conceptually: what do the U matrices do in LSTMs?

In an LSTM, every gate is influenced by **two things**:

1. the current input x_t (via W_*)
2. the previous hidden state h_{t-1} (via $U*$)

So, **the U matrices are the "recurrent/context" weights**: they decide how the model's *current context* (stored in h_{t-1}) should affect each gate's decision.

Input gate: U_i — "should we write new info?"

Input gate:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i).$$

- **Role of U_i:** uses the context h_{t-1} to control **how much new information** gets written into memory.
- Intuition: "Given what I already know, do I need to update memory now?"

Forget gate: U_f — "should we keep or erase memory?"

Forget gate:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f).$$

- **Role of U_f:** uses the context h_{t-1} to decide **what to retain** from the previous cell state c_{t-1}.
- Intuition: "Given what I've been tracking, is the old memory still relevant or should I drop it?"

Output gate: U_o — "what part of memory should I reveal?"

Output gate:

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o).$$

- **Role of U_o:** uses the context h_{t-1} to decide **how much of the cell state** should be exposed as the hidden state output h_t.
- Intuition: "Given the current context, how much should I show to the outside world right now?"

Summary

The U matrices make the gates **context-aware**: they tell each gate how to use the previous hidden state h_{t-1} to decide **write** (U_i), **keep/erase** (U_f), and **reveal** (U_o).

Backpropagation Through Time (BPTT)

For an RNN/LSTM, the hidden state is computed **sequentially**:

$$h_t = f_\theta(h_{t-1}, x_t), \quad t = 1, \dots, T.$$

For next-token prediction, a common loss is:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T-1} \ell_t(\theta), \quad \ell_t(\theta) = -\log p_\theta(x_{t+1} \mid x_{\le t}).$$

Backpropagation Through Time (BPTT) means we compute gradients of the total loss with respect to parameters by applying the chain rule **through the unrolled time steps**:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^{T-1} \frac{\partial \ell_t}{\partial \theta}.$$

Because each h_t depends on earlier states, each term includes indirect dependencies via the recurrence. A useful way to see this is:

$$\frac{\partial \ell_t}{\partial \theta} = \sum_{k=1}^{t} \frac{\partial \ell_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_k}{\partial \theta}.$$

So gradients propagate backward along time:

$$\ell_t \to h_t \to h_{t-1} \to \cdots \to h_1.$$

Practical notes

- BPTT can suffer from **vanishing/exploding gradients** when

  $$\left\| \frac{\partial h_t}{\partial h_{t-1}} \right\|$$

  repeatedly shrinks or grows across many steps.
- Common stabilizers: **gradient clipping**, gated units (LSTM/GRU), and **truncated BPTT** (backprop only across a fixed window of time steps).

What is a Transformer?

A Transformer primary role (at least in its internal layers) is to compute a **contextualized representation** for every token position.

Start with a token sequence:

$$x_1, x_2, \ldots, x_T$$

After embedding (token + position), we have initial vectors:

$$h_1^{(0)}, h_2^{(0)}, \ldots, h_T^{(0)}.$$

A Transformer applies multiple layers. Each layer updates every token's vector by mixing in information from other tokens (via **attention**):

$$h_t^{(\ell+1)} = \mathrm{Block}\left(h_1^{(\ell)}, \ldots, h_T^{(\ell)}\right)_t.$$

So after L layers, each token has a final contextual vector:

$$h_t^{(L)} \in \mathbb{R}^d.$$

What does "contextual" mean here?

It means $h_t^{(L)}$ is not just "the embedding of token t", but a representation that depends on the entire sentence:

$$h_t^{(L)} = f_\theta(x_1, x_2, \ldots, x_T; t).$$

For example, the vector for "bank" will be different in:

- $*$ "river bank"
- $*$"bank account"

because attention allows the token "bank" to incorporate information from nearby words ("river" vs "account").

Autoregressive (GPT-style) vs bidirectional (BERT-style)

- **GPT / decoder-only:** token t can only attend to $j \leq t$ (past context).
- **BERT / encoder:** token t can attend to all $j \in \{1, \ldots, T\}$ (full context).

Summary

A Transformer repeatedly applies attention + MLP updates so that every token's vector becomes a **context-dependent ("contextual") meaning representation** that is useful for the final task (next-token prediction, classification, etc.).

Transformers: Architecture

Transformers became the dominant sequence modeling architecture because they address two core bottlenecks of LSTMs:

1. **Sequential computation** (slow training/inference for long sequences)
2. **Long-range dependency learning** (hard credit assignment across many steps)

The Transformer replaces **recurrence** with **self-attention**, enabling strong performance and **excellent scaling** with data/compute.

LSTM bottlenecks

Inherently sequential training

An LSTM computes states step-by-step:

$$(h_t, c_t) = \mathrm{LSTM}(x_t, h_{t-1}, c_{t-1}), \quad t = 1, \ldots, T.$$

You cannot compute h_t until you have h_{t-1}, so training has limited parallelism over time.

Long-range dependencies have long gradient paths

Even with gates, the influence of x_1 on the loss at time t must flow through many intermediate states:

$$x_1 \to (h_1, c_1) \to (h_2, c_2) \to \cdots \to (h_t, c_t) \to \ell_t.$$

This creates a long credit-assignment chain (still challenging at scale).

Key Transformer ideas

Key idea #1: Self-attention instead of recurrence

At a layer, each position can directly mix information from other positions.

Given hidden states $X \in \mathbb{R}^{T\times d}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

Scaled dot-product attention:

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right), \quad \mathrm{Att}(X) = AV.$$

- In **autoregressive** Transformers (GPT-style), M is a **causal mask**:

$$M_{t,u} = \begin{cases} 0, & u \le t \\ -\infty, & u > t \end{cases}.$$

This is important because: token t can attend directly to token $t-200$ in one hop.

Key idea #2: Multi-head attention

Instead of one attention, use H heads (similar to multiple filters in CNNs):

$$\mathrm{head}_h = \mathrm{Att}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)}),$$

$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)W_O.$$

This is important because: different heads specialize (syntax, coreference, local patterns, global topics, etc.).

Key idea #3: Parallel training over sequence positions

With causal masking (or bidirectional masking for encoders), Transformers compute all positions in parallel in a forward pass:

$$X^{(l+1)} = \mathrm{Block}(X^{(l)}),$$

rather than step-by-step recurrence. This maps extremely well to GPUs/TPUs.

This is important because:: training scales efficiently to huge datasets and models.

Key idea #4: Positional information is explicit

Because attention is permutation-invariant, Transformers add **positional information**:

- Learned positional embeddings:

$$x_t^{(0)} = W_e[s_t] + W_p[t].$$

- Or sinusoidal positional encodings (original Transformer).

This is important because:: the model learns order **without recurrence**.

Key idea #5: Residual connections + LayerNorm stabilize deep networks

A typical pre-LN block:

$$X' = X + \mathrm{MHA}(\mathrm{LN}(X)),$$

$$X^{\mathrm{next}} = X' + \mathrm{FFN}(\mathrm{LN}(X')).$$

This is important because:: enables much deeper models and more stable optimization.

Key idea #6: The FFN (MLP) expands capacity per token

Per position:

$$\mathrm{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2,$$

commonly with **GELU**. This gives a powerful nonlinearity at each token position.

Why Transformers are better in practice

Better long-range modeling

Self-attention creates short paths:

- LSTM path length from token i to token t is $O(t-i)$
- Transformer path length (within a layer) is $O(1)$ because attention can connect any pair directly

This often improves tasks needing long context (documents, code, dialogue, retrieval-like behavior).

Faster training due to parallelism

Even though self-attention is $O(T^2)$ in sequence length, it is highly parallel and typically faster than sequential recurrence for moderate-to-long contexts.

Better scaling laws and transfer

Transformers scale smoothly with:

- model size
- data size
- compute

and yield strong generalization and “few-shot” style transfer when trained autoregressively on diverse data.

Important caveat (where LSTMs can still win)

For very long sequences with tight compute/memory:

- Standard attention costs $O(T^2)$ memory/time.
- LSTMs cost $O(T)$ and may be more efficient for certain streaming or embedded settings.

This is why efficient attention, state-space models, and hybrids exist.

Summary

Transformers won because **self-attention + parallel training + stable deep optimization** made it easier to learn long-range structure and scale to large datasets and models, outperforming LSTMs on most modern sequence tasks.

Transformer: Why we need “Attention”

Consider the sentence:

> **“the pink elephant tried to get into the car but it was too …”**

When you reach the word **“too”**, you (as a human) immediately know you must look back in the sentence to decide:

- **too what?** too *big*? too *small*? too *old*?
- what is “it”? does **it** refer to the **car** or the **elephant**?
- what earlier word(s) provide the key information to interpret “too”?

The core problem: not all past words matter equally

At the moment you process **“too”**, the most useful tokens are likely:

- **“car”** (it was too ___ to get into the car)
- **“elephant”** (an elephant is big; maybe the elephant is too big)
- **“it”** (coreference: what does “it” refer to?)

But many tokens are less important:

- “the”, “pink”, “tried”, “to”, “get”, “into”, “but”, “was”

So we want the model to do something like:

> “When computing the representation of **too**, pay *more* attention to **car / elephant / it**, and *less* to the rest.”

Why LSTMs/RNNs struggle here

In an RNN/LSTM, information from earlier tokens must be compressed into a single evolving state:

$$(h_t, c_t) = \mathrm{LSTM}(x_t, h_{t-1}, c_{t-1}).$$

So by the time we reach “too”, the model must have **carried** the relevant facts about “car” and “elephant” through many steps. This is possible, but it is indirect and can be difficult for long sequences.

What attention adds

Attention gives a direct mechanism:

> “At position t, look back at all earlier positions $1..t$, decide which ones are relevant, then mix their information.”

This is like having a **soft search-and-retrieve** over the sentence.

What “self-attention” means

- **Attention** means “use a query to retrieve information from a set of (key, value) pairs.”
- **Self-attention** means the query, keys, and values all come from the **same sequence**.

So in our sentence:

- the token **“too”** creates a **query**
- every token in the sentence provides **keys** and **values**
- “too” uses its query to decide how much to use each token’s value

What are the **keys** and **values** in self-attention?

In **self-attention**, the query, keys, and values all come from the **same sentence**.

Assume the sentence is tokenized into T tokens, with embeddings:

$$x_1, x_2, \ldots, x_T, \quad x_j \in \mathbb{R}^{d_e}.$$

Which token provides the **query**?

If we are computing the contextual representation for the token at position t (here the word **“too”**), then:

The **query** comes from the current token's embedding x_t:

$q_t = x_t W_Q \in \mathbb{R}^{d_k}$.

So: **"too" provides the query** (because we are asking: what context does "too" need?). **Query** is the target *token* for which we need to know the *Context* for.

Which tokens provide the **keys** and **values**?

Every token position j in the sentence provides:

- a **key** vector k_j
- a **value** vector v_j

computed from that token's embedding x_j:

$$k_j = x_j W_K \in \mathbb{R}^{d_k}, \quad v_j = x_j W_V \in \mathbb{R}^{d_v}.$$

So:

- The token **"car"** provides k_{car} and v_{car}
- The token **"elephant"** provides k_{elephant} and v_{elephant}
- The token **"it"** provides k_{it} and v_{it}
- Even function words like **"the"** provide their own k_j, v_j (usually they receive small weights)

> **Important:** A "key" is **not** "the previous token."
> A key is a **learned projection of each token's embedding** that acts like an "address/label" for matching.

key = "What do I contain/what am I about?"

For the current token at position t, we have a **query** q_t. The relevance score is:

$s_{t,j} = \frac{q_t \cdot k_j}{\sqrt{d_k}}$.

What is this score, intuitively?

- The dot product $q_t \cdot k_j$ is a **similarity** measure:
  - If q_t and k_j point in a similar direction, the dot product is **large and positive**.
  - If they are unrelated/orthogonal, the dot product is near **0**.
  - If they point in opposite directions, it can be **negative**.

So you can think of it as:

> "How well does token j match what token t is looking for **regardless of the context of the sentence**?"

Why divide by $\sqrt{d_k}$?

If the components of q_t and k_j are roughly random with variance around 1, then the dot product magnitude grows with dimension:

$$q_t \cdot k_j = \sum_{r=1}^{d_k} q_{t,r} k_{j,r},$$

so its typical scale grows like $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ keeps scores in a stable numeric range across different head dimensions:

$$s_{t,j} = \frac{q_t \cdot k_j}{\sqrt{d_k}} \approx O(1).$$

This prevents the softmax (done at a later stage) from saturating too early (i.e., producing extremely peaky weights) when d_k is large.

value = "If you attend to me, what information should you take?"

Within a layer, token t forms a weighted combination of other tokens' **value vectors**:

$y_t = \sum_{j=1}^{T} \alpha_{t,j}\, v_j, \quad \alpha_{t,j} = \mathrm{softmax}\left(\frac{q_t \cdot k_j}{\sqrt{d_k}}\right).$

This is the mechanism by which token t pulls in the information it needs from other tokens, producing an updated, more contextual representation.

The value v_j is the **information payload** that will be mixed into the output:

So **values are what get averaged**, keys are what get **matched**.

Softmax converts scores into weights (so we can average). The scores $s_{t,j}$ are just real numbers (can be negative, don't sum to 1). Softmax converts them into **nonnegative weights that sum to 1**:

$$\alpha_{t,j} = \frac{\exp(s_{t,j})}{\sum_u \exp(s_{t,u})}.$$

This is why we can interpret $\alpha_{t,j}$ as "how much attention token t gives to token j".

In the sentence example ("... it was too ...")

When computing the representation of **"too"** (position t):

- **Query**: q_t comes from **"too"**
- **Keys/Values**: each prior word contributes k_j, v_j
- If "car" is most relevant, then $\alpha_{t,\text{car}}$ is large, so v_{car} strongly influences the contextual output y_t.

Intuitively:

- The query for "too" asks: *"What earlier context helps interpret me?"*
- Keys answer: *"Match me if I'm relevant."*
- Values provide: *"Here is the information you should incorporate if you match me."*

Autoregressive (GPT-style)

In GPT-style self-attention, "too" can attend only to **previous** tokens:

$$j \leq t.$$

So in that setting:

- keys/values come from tokens at positions $1, 2, \ldots, t$
- future tokens $j > t$ are masked out (not "previous token only"—**all previous tokens**).

The role of **value**

We are computing a contextual representation for the token at position t (the word **“too”**) in the sentence:

> **“the pink elephant tried to get into the car but it was too …”**

At this moment, “too” needs context to decide what comes next (e.g., *too big*). The model’s job is to create a vector y_t that summarizes the **most relevant earlier information** for interpreting “too”.

What does “value” represent for each token?

Each token j has an embedding $x_j \in \mathbb{R}^{d_e}$.
Self-attention forms three projections:

$$k_j = x_j W_K, \quad v_j = x_j W_V, \quad q_t = x_t W_Q.$$

- The **key** k_j is used for **matching** or **score** as shown earlier (to compute relevance to the query).
- The **value** v_j is the *thing we actually take from token* j if we decide it matters.

That is why we call v_j as the **information contribution of a token**:

> If token j is important for token t, then we want token j to **contribute information** to the representation of token t.
> That contribution is the vector v_j.

Why is v_j a vector?

In a neural language model, each token is represented not as a single number, but as a **feature vector**. A token is not just “car” — it has many properties

Take the word **“car”** in the sentence:

> “… tried to get into the **car** but it was too …”

To predict what comes after **“too”**, the model may need multiple kinds of information about “car”, such as:

- “car” is a **container / object with an interior**
- “car” has a notion of **size / capacity**
- “car” is related to **getting into**
- “car” is a **noun**, object of a verb, etc.

A single scalar cannot represent all these simultaneously. A vector can. So we represent token x_j by a vector $v_j \in \mathbb{R}^{d_e}$ whose coordinates are different learned features.

The value vector is a learned linear transform of the token representation:

$$v_j = x_j W_V \in \mathbb{R}^{d_v}.$$

Each component of v_j can be thought of as a learned feature, e.g.:

$$v_j = \begin{bmatrix} (\text{size-related feature}) \\ (\text{container/capacity feature}) \\ (\text{noun/object feature}) \\ \vdots \end{bmatrix}.$$

This is not manually designed—training learns these features because they help minimize prediction loss.

Concrete sentence-based example

When processing **“too”**, suppose the model decides the **important earlier** word is **“car”**.

- “car” might carry **information like: *a container with limited size/entry constraint*
- “elephant” might carry information like: *very large animal*

The model needs a vector representing these facts to help predict what comes after “too”.

So the **value vectors** are the learned way each word says:

> “If you attend to me, here is the **information** you should take from me.”

Importantly, v_j is **not the raw word**. It is a learned vector that can encode features useful for prediction.

Why do we mix values with a weighted sum?

We want y_t to be a combination of the relevant information from the past.

So we compute:

$$y_t = \sum_{j=1}^{t} \alpha_{t,j}\, v_j.$$

- If "car" is very relevant, $\alpha_{t,\text{car}}$ should be large, so v_{car} contributes a lot.
- If "pink" is irrelevant, $\alpha_{t,\text{pink}}$ should be small, so v_{pink} contributes almost nothing.

This is exactly like taking a **weighted average** of the value vectors.

Where do the weights come from?

We first compute raw relevance scores:

$s_{t,j} = \frac{q_t \cdot k_j}{\sqrt{d_k}}.$

These are just real numbers. They can be negative or positive, and they do **not** automatically sum to 1.

But to form a stable weighted mixture, we want weights that:

1. are **nonnegative** (so "importance" is not negative),
2. sum to **1** (so it behaves like a weighted average),
3. smoothly increase when a score increases.

The standard function that does exactly this is **softmax**.

What does softmax do and why is it used?

Given the vector of scores for fixed t:

$s_{t,1}, s_{t,2}, \ldots, s_{t,t},$

softmax turns them into weights:

$\alpha_{t,j} = \frac{\exp(s_{t,j})}{\sum_{u=1}^{t} \exp(s_{t,u})}.$

Now:

- $\alpha_{t,j} \geq 0$
- $\sum_{j=1}^{t} \alpha_{t,j} = 1$

So $\alpha_{t,j}$ can be interpreted as:

> "How much attention token t pays to token j."

Small numeric example

Suppose when computing the representation of "too" (position t), the model produces these scores:

- token "car": $s_{t,\text{car}} = 2.0$
- token "elephant": $s_{t,\text{elephant}} = 1.0$
- token "pink": $s_{t,\text{pink}} = 0.0$

Softmax gives:

$$\alpha_{t,\text{car}} = \frac{e^2}{e^2 + e^1 + e^0} \approx 0.665,$$

$$\alpha_{t,\text{elephant}} \approx 0.245, \quad \alpha_{t,\text{pink}} \approx 0.090.$$

So the output becomes:

$$y_t \approx 0.665\, v_{\text{car}} + 0.245\, v_{\text{elephant}} + 0.090\, v_{\text{pink}}.$$

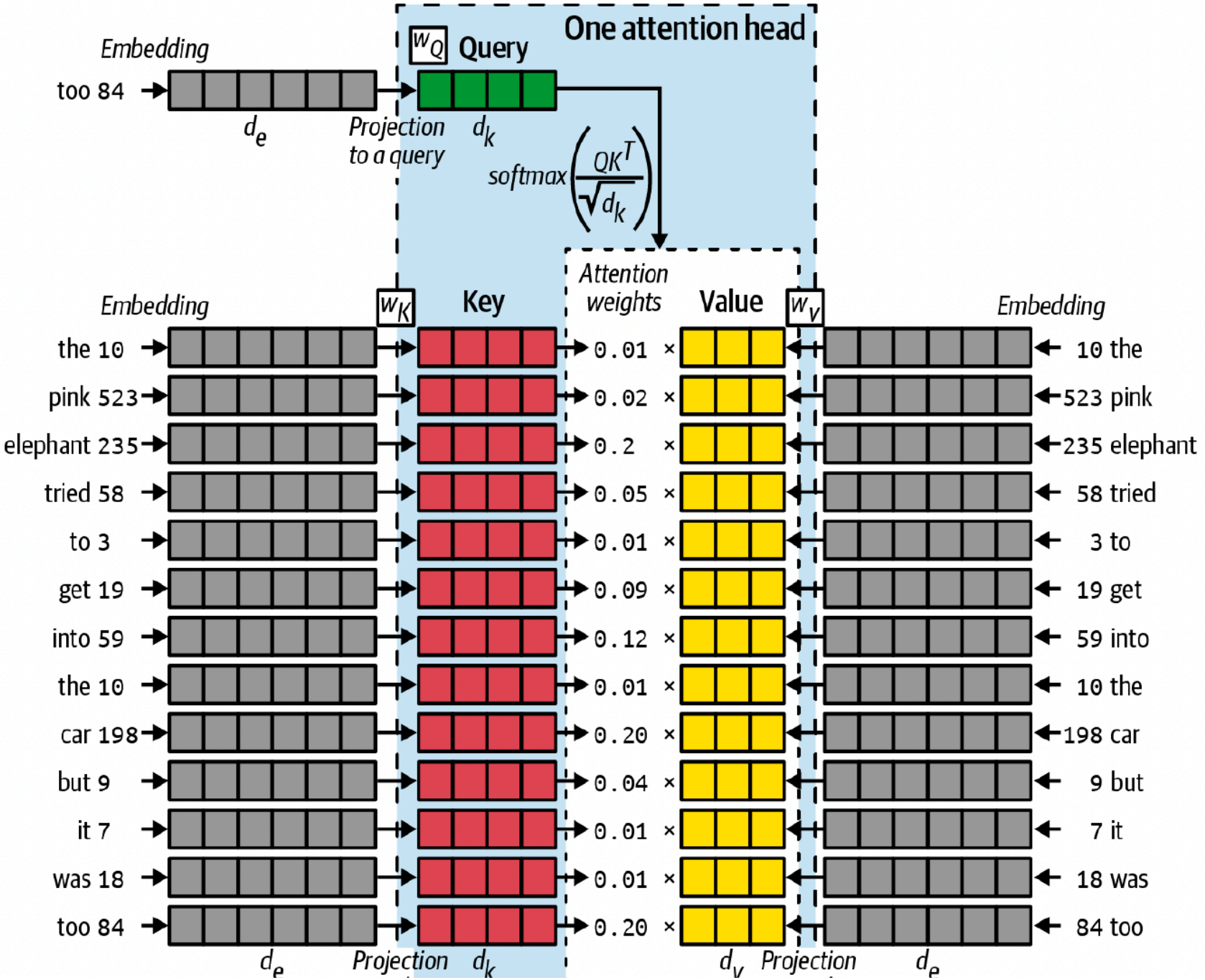
Interpretation:

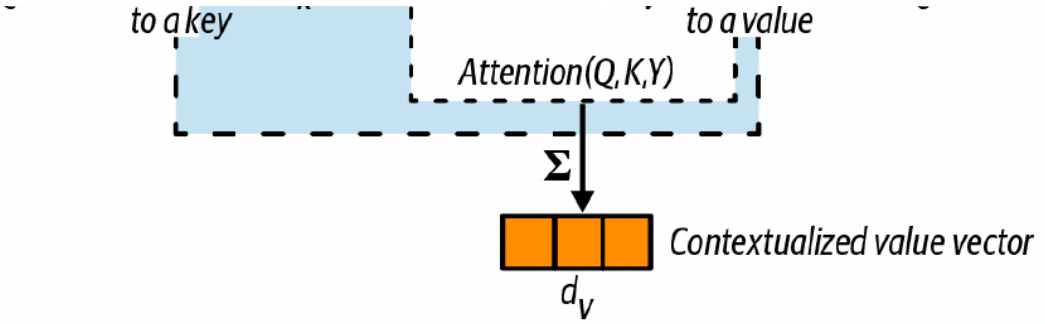
- y_t is dominated by the “car” information, with some “elephant” information mixed in.
- This combined context vector y_t is then used by the next layers to help predict what comes next (likely *big*).

Summary

- **Keys** k_j are used to compute match scores with the query (relevance).
- **Softmax** converts those scores into normalized, nonnegative weights.
- **Values** v_j are the information vectors that are mixed (weighted averaged) to create the contextual representation: $y_t = \sum_{j=1}^{t} \alpha_{t,j} v_j$.

Transformer Architecture


One attention head
Embedding
w_Q
Query
too 84
d_e
Projection to a query
d_k
$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$
Attention weights
Embedding
w_K
Key
Value
w_V
Embedding
the 10
pink 523
elephant 235
tried 58
to 3
get 19
into 59
the 10
car 198
but 9
it 7
was 18
too 84
0.01 ×
0.02 ×
0.2 ×
0.05 ×
0.01 ×
0.09 ×
0.12 ×
0.01 ×
0.20 ×
0.04 ×
0.01 ×
0.01 ×
0.20 ×
10 the
523 pink
235 elephant
58 tried
3 to
19 get
59 into
10 the
198 car
9 but
7 it
18 was
84 too
d_e
Projection
d_k
d_v
Projection
d_e



Understanding "Attention" using the diagram (one attention head)

> **Goal:** compute a *contextualized representation* of the current token (here the last token **"too"**) by looking back at all tokens in the sequence and deciding **which ones matter most**.

Start with token embeddings (gray blocks)

Each token in the sentence (e.g., "the", "pink", "elephant", ..., "too") is mapped to an **embedding vector**:

- Embedding dimension: d_e
- Token embedding for position j: $x_j \in \mathbb{R}^{d_e}$

In the diagram, the gray bars are these vectors x_j each representing an embedding for a token.

The model learns three projection matrices: W_Q, W_K, W_V

A single attention head learns **three** linear maps:

- $W_Q \in \mathbb{R}^{d_e \times d_k}$ (Query projection)
- $W_K \in \mathbb{R}^{d_e \times d_k}$ (Key projection)
- $W_V \in \mathbb{R}^{d_e \times d_v}$ (Value projection)

What each learns:

- W_Q learns *what the current token is looking for* (a “search vector”).
- W_K learns *how each token advertises what it contains* (an “address / label”).
- W_V learns *what **information** each token will contribute if selected* (the “content payload”).

Build Query, Keys, Values (colored blocks)

For the **current token** (here “too”), we create a **query**:

$q = x_t W_Q \in \mathbb{R}^{d_k}$.

For **every token** in the sequence, we create a **key** and a **value**:

$$k_j = x_j W_K \in \mathbb{R}^{d_k}, \quad v_j = x_j W_V \in \mathbb{R}^{d_v}.$$

In the diagram:

- **Green** = query vector q
- **Red** = key vectors k_j
- **Yellow** = value vectors v_j

Compute “relevance” scores with dot products

The attention head scores how relevant each token j is to the current query:

$$s_j = \frac{q \cdot k_j}{\sqrt{d_k}}.$$

In matrix form (for a whole sequence at once for all queries and keys):

$$S = \frac{QK^\top}{\sqrt{d_k}}.$$

Convert scores into **attention weights** (using softmax)

We normalize the scores across all tokens:

$$\alpha_j = \frac{\exp(s_j)}{\sum_{u=1}^{T} \exp(s_u)}.$$

So:

- $\alpha_j \geq 0$
- $\sum_{j=1}^{T} \alpha_j = 1$

In the diagram, these are the numbers like **0.01, 0.02, 0.20, ...** next to each token.

Interpretation: larger α_j means the model considers token j more relevant for understanding "too".

Compute the contextualized output as a weighted sum of Values

Finally, the head produces an output vector (the orange block at the bottom):

$$y = \sum_{j=1}^{T} \alpha_j v_j \in \mathbb{R}^{d_v}.$$

In matrix form:

$$A = \mathrm{softmax}(S), \quad Y = AV.$$

Stack all value vectors as rows of a matrix:

$$V = \begin{bmatrix} v_1^\top \\ v_2^\top \\ \vdots \\ v_T^\top \end{bmatrix} \in \mathbb{R}^{T \times d_v}.$$

The score matrix representing score for each token against the other:

$S \in \mathbb{R}^{T \times T}$.

Apply softmax **row-wise** to get the attention weight matrix:

$$A = \mathrm{softmax}(S), \quad A \in \mathbb{R}^{T \times T}.$$

Row t of A is the distribution over tokens that token t attends to:

$$A_{t,:} = (\alpha_{t,1}, \ldots, \alpha_{t,T}).$$

Then the outputs for **all positions** are:

$Y = AV \in \mathbb{R}^{T \times d_v}$.

- If $A_{t,j}$ is large, token j has a strong influence on token t's updated representation.
- If $A_{t,j}$ is near 0, token j contributes almost nothing to token t.

Key intuition:

- **Keys** decide *where* to look (matching)
- **Values** decide *what* you get from each location (content)

(If autoregressive / GPT-style) causal masking

In a GPT-like model generating left-to-right, the token at position t must not look at future tokens $> t$. We enforce:

$$s_j = \begin{cases} \frac{q \cdot k_j}{\sqrt{d_k}}, & j \le t \\ -\infty, & j > t \end{cases} \quad \Rightarrow \quad \alpha_j = \mathrm{softmax}(s)_j.$$

So "too" only attends to words that came before it.

Multi-head attention (big picture)

A real Transformer uses multiple heads. Each head has its own projections:

$$W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \quad \text{for } h = 1, \ldots, H.$$

Each head outputs

$$y^{(h)}$$

, then:

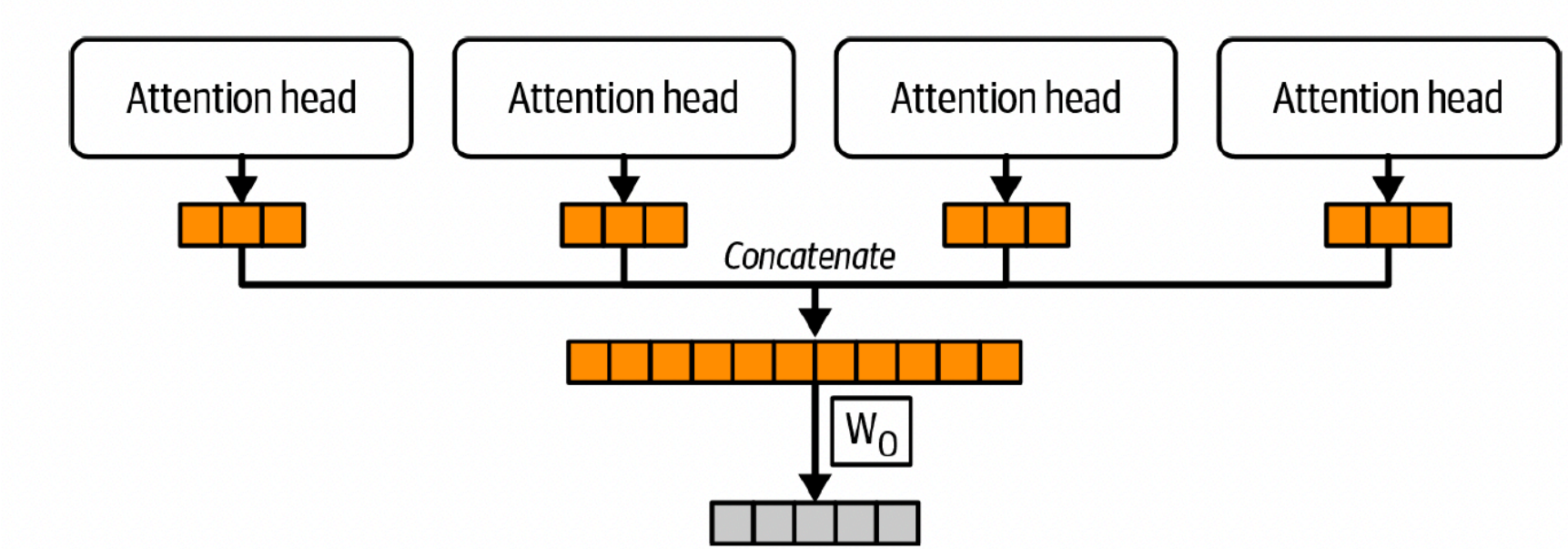
$$\mathrm{MHA}(X) = \mathrm{Concat}(y^{(1)}, \ldots, y^{(H)})W_O.$$

Different heads can specialize in different relations (coreference, modifiers, syntax, long-range links, etc.).

One-sentence takeaway

Self-attention lets the token “too” directly look back at the entire sentence, assign importance weights to earlier tokens (like “car”, “elephant”, “it”), and combine their information into a single contextualized vector.

Multihead Attention



Source:

Multi-head attention lets the model attend to the sequence in **multiple different ways at the same time**. Each head has its own learned projections, so different heads can specialize in different relations (e.g., local syntax, long-range coreference, etc.).

Setup and shapes

Let the input to attention be:

$X \in \mathbb{R}^{T \times d}$,

where:

- T = sequence length
- d = model dimension

Choose number of heads H, and per-head dimensions:

$d_k = d_v = \frac{d}{H}$.

(So concatenating all heads returns to dimension d.)

One head (head h)

Each head has its own projection matrices:

$$W_Q^{(h)} \in \mathbb{R}^{d \times d_k}, \quad W_K^{(h)} \in \mathbb{R}^{d \times d_k}, \quad W_V^{(h)} \in \mathbb{R}^{d \times d_v}.$$

Compute:

$$Q^{(h)} = XW_Q^{(h)}, \quad K^{(h)} = XW_K^{(h)}, \quad V^{(h)} = XW_V^{(h)}.$$

Scores:

$$S^{(h)} = \frac{Q^{(h)}(K^{(h)})^\top}{\sqrt{d_k}} \in \mathbb{R}^{T \times T}.$$

(For autoregressive / GPT-style attention, add a causal mask M before softmax.)

Attention weights:

$$A^{(h)} = \mathrm{softmax}(S^{(h)} + M) \in \mathbb{R}^{T \times T}, \quad A_{t,:}^{(h)} = \mathrm{softmax}\big((S^{(h)} + M)_{t,:}\big).$$

Head output:

$$Y^{(h)} = A^{(h)} V^{(h)} \in \mathbb{R}^{T \times d_v}.$$

Combine heads (concatenate + output projection)

Concatenate all heads along the feature dimension:

$$Y_{\text{cat}} = \mathrm{Concat}(Y^{(1)}, \ldots, Y^{(H)}) \in \mathbb{R}^{T \times (Hd_v)}.$$

Since $Hd_v = d$, this is typically:

$Y_{\text{cat}} \in \mathbb{R}^{T \times d}$.

Then apply a final learned output projection:

$$W_O \in \mathbb{R}^{d \times d}, \quad \text{MHA}(X) = Y_{\text{cat}} W_O \in \mathbb{R}^{T \times d}.$$

Compact formula for multi-head attention

$$\text{MHA}(X) = \text{Concat}\Big(\text{Att}(XW_Q^{(1)}, XW_K^{(1)}, XW_V^{(1)}), \ldots, \text{Att}(XW_Q^{(H)}, XW_K^{(H)}, XW_V^{(H)})\Big) W_O,$$

where:

$$\text{Att}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

Why multiple heads help

- A single attention head produces one set of weights per token: it can focus on only one “pattern” at a time.
- Multiple heads allow the model to compute **multiple different attention patterns in parallel**.
- After concatenation, the model keeps all these perspectives and can combine them with W_O.

Summary

Multi-head attention runs several attention mechanisms in parallel, each with its own learned projections, then concatenates their outputs to produce a richer contextual representation.

Example: A 3-token walk-through of **self-attention**

From the earlier sentence (and picture), we consider only **three tokens**:

1. **elephant**
2. **car**
3. **too** (this is the token whose context we want to compute)

In the picture:

- **Query** (green) comes from the current token ("too")
- **Keys** (red) come from *every token* in the sequence
- **Values** (yellow) come from *every token* in the sequence
- **Softmax weights** are the attention weights shown next to tokens
- The output is the **weighted sum of value vectors** (orange)

Step 0 — Embeddings (gray blocks)

Let the embedding dimension be $d_e = 2$ (tiny for hand calculation).

We choose simple embeddings (rows correspond to tokens, columns are embeddings):

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ x_3^\top \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \quad \text{where } x_1 = \text{elephant},\ x_2 = \text{car},\ x_3 = \text{too}.$$

> (In a real Transformer, X would be token embeddings **plus** positional embeddings, but we keep it simple here.)

Step 1 — Project to Queries / Keys / Values (green/red/yellow in the picture)

A single attention head learns three matrices:

$$W_Q \in \mathbb{R}^{d_e \times d_k}, \quad W_K \in \mathbb{R}^{d_e \times d_k}, \quad W_V \in \mathbb{R}^{d_e \times d_v}.$$

For this tiny example, set $d_k = d_v = 2$ and use the identity projections (to keep arithmetic simple):

$W_Q = W_K = W_V = I_2$.

So:

$$Q = XW_Q = X, \quad K = XW_K = X, \quad V = XW_V = X.$$

Step 2 — Compute similarity scores (the $QK^\top/\sqrt{d_k}$ box in the picture)

Scaled dot-product score matrix:

$$S = \frac{QK^\top}{\sqrt{d_k}}, \quad d_k = 2, \quad \sqrt{d_k} = \sqrt{2}.$$

First compute $QK^\top = XX^\top$:

$$XX^\top = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Now scale by $\sqrt{2}$:

$$S = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.707 & 0 & 0 \\ 0 & 0.707 & 0.707 \\ 0 & 0.707 & 0.707 \end{bmatrix}.$$

Interpretation:

- Row t = scores produced by **query** q_t against all **keys** k_j.
- For token 3 (“too”), row 3 shows it matches tokens 2 and 3 more than token 1.

Step 3 — (Autoregressive) causal mask (GPT-style)

If we are generating left-to-right, token t can only attend to positions $j \le t$.

Mask entries above the diagonal to $-\infty$:

$$S_{\text{masked}} = \begin{bmatrix} 0.707 & -\infty & -\infty \\ 0 & 0.707 & -\infty \\ 0 & 0.707 & 0.707 \end{bmatrix}.$$

Step 4 — Softmax to get attention weights (the numbers next to tokens in the picture)

Row-wise softmax:

$A = \text{softmax}(S_{\text{masked}}).$

Compute each row:

- Row 1: softmax over $[0.707]$ gives

$$A_{1,:} = [1,\ 0,\ 0].$$

- Row 2: softmax over $[0,\ 0.707]$

$$e^0 = 1, \quad e^{0.707} \approx 2.028, \quad \text{sum} \approx 3.028$$

$$A_{2,:} \approx [0.330,\ 0.670,\ 0].$$

- Row 3: softmax over $[0,\ 0.707,\ 0.707]$

$$[1,\ 2.028,\ 2.028], \quad \text{sum} \approx 5.056$$

$$A_{3,:} \approx [0.198,\ 0.401,\ 0.401].$$

So the attention weight matrix is approximately:

$$A \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.330 & 0.670 & 0 \\ 0.198 & 0.401 & 0.401 \end{bmatrix}.$$

This is where “keys are matched” shows up:
the keys only affected the **scores** S and therefore the **weights** A.

Step 5 — Weighted sum of **values** (the AV box in the picture)

The output of attention is:

$$Y = AV.$$

Since $V = X$ here:

- Token 1 output:

$$y_1 = 1 \cdot v_1 = [1, 0]$$

- Token 2 output:

$$y_2 \approx 0.330\, v_1 + 0.670\, v_2 \approx 0.330[1, 0] + 0.670[0, 1] = [0.330,\ 0.670]$$

- Token 3 (“too”) output:

$$y_3 \approx 0.198\, v_1 + 0.401\, v_2 + 0.401\, v_3$$

$$\approx 0.198[1, 0] + 0.401[0, 1] + 0.401[0, 1] = [0.198,\ 0.802].$$

This is where “values are averaged” shows up:
the values v_j are the vectors being mixed using weights $\alpha_{t,j} = A_{t,j}$.

Interpreting the result for the picture/sentence

For token 3 (“too”), the weights were:

$$\alpha_{3,1} \approx 0.198\ (\text{elephant}), \quad \alpha_{3,2} \approx 0.401\ (\text{car}), \quad \alpha_{3,3} \approx 0.401\ (\text{too itself}).$$

So “too” builds a contextual vector mostly from **car** (and itself), and less from **elephant**.

In a trained model on the full sentence:

- the “car” value vector v_{car} can encode “container / size constraint”
- the “elephant” value vector v_{elephant} can encode “large object”

- the mixture y_{too} then helps the network predict what follows “too” (e.g., *big*).

Where this sits inside a Transformer block (big picture)

A Transformer layer for the full sequence is typically:

1. **Self-attention:** $Y = \mathrm{Attention}(X)$ (what we just computed)
2. **Residual + LayerNorm:** $X' = X + Y$ (plus LN in real models)
3. **Feed-forward (MLP):** $Z = \mathrm{FFN}(X')$
4. **Residual + LayerNorm:** $X^{\text{next}} = X' + Z$

Stacking layers repeatedly refines each token’s **contextual meaning** vector.

Matrix Form Summary of a Transformer

Assume a sequence of length T with token ids:

$$s = (s_1, \ldots, s_T), \quad s_t \in \{1, \ldots, V\}.$$

Let model dimension be d, head key dimension d_k, and value dimension d_v.

Token + positional embeddings

Token embedding matrix:

$$W_e \in \mathbb{R}^{V \times d}.$$

Positional embedding matrix (learned):

$$W_p \in \mathbb{R}^{T \times d}.$$

Construct the embedding matrix $X \in \mathbb{R}^{T \times d}$:

$$X = \mathrm{Embed}(s) + W_p,$$

where $\mathrm{Embed}(s)$ stacks rows $W_e[s_t]$:

$$\mathrm{Embed}(s) = \begin{bmatrix} W_e[s_1] \\ \vdots \\ W_e[s_T] \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

So each row X_t is the initial representation of token t.

Linear projections to Queries / Keys / Values

Learned projection matrices:

$$W_Q \in \mathbb{R}^{d \times d_k}, \quad W_K \in \mathbb{R}^{d \times d_k}, \quad W_V \in \mathbb{R}^{d \times d_v}.$$

Compute:

$$Q = XW_Q \in \mathbb{R}^{T \times d_k}, \quad K = XW_K \in \mathbb{R}^{T \times d_k}, \quad V = XW_V \in \mathbb{R}^{T \times d_v}.$$

Scaled dot-product attention scores

Compute the score matrix:

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{T \times T}.$$

Causal masking (autoregressive / GPT-style)

Define a mask

$$M \in \mathbb{R}^{T \times T}$$

:

$$M_{t,j} = \begin{cases} 0, & j \le t \\ -\infty, & j > t \end{cases}.$$

Apply it to the scores:

$$\tilde{S} = S + M.$$

(If you are not autoregressive, skip the mask and use S directly.)

Softmax to get attention weights

Row-wise softmax:

$$A = \mathrm{softmax}(\tilde{S}) \in \mathbb{R}^{T \times T}, \quad A_{t,:} = \mathrm{softmax}(\tilde{S}_{t,:}).$$

So each row $A_{t,:}$ is a distribution over which tokens token t attends to:

$$A_{t,j} \geq 0, \quad \sum_{j=1}^{T} A_{t,j} = 1.$$

Weighted sum of values (the AV step)

The attention output is:

$Y = AV \in \mathbb{R}^{T \times d_v}$.

Row t is:

$y_t = \sum_{j=1}^{T} A_{t,j}\, v_j$.

Multi-head attention (compact form)

For head $h = 1, \ldots, H$:

$$Q^{(h)} = XW_Q^{(h)}, \quad K^{(h)} = XW_K^{(h)}, \quad V^{(h)} = XW_V^{(h)},$$

$$Y^{(h)} = \mathrm{softmax}\left(\frac{Q^{(h)}(K^{(h)})^\top}{\sqrt{d_k}} + M\right) V^{(h)}.$$

Concatenate and project:

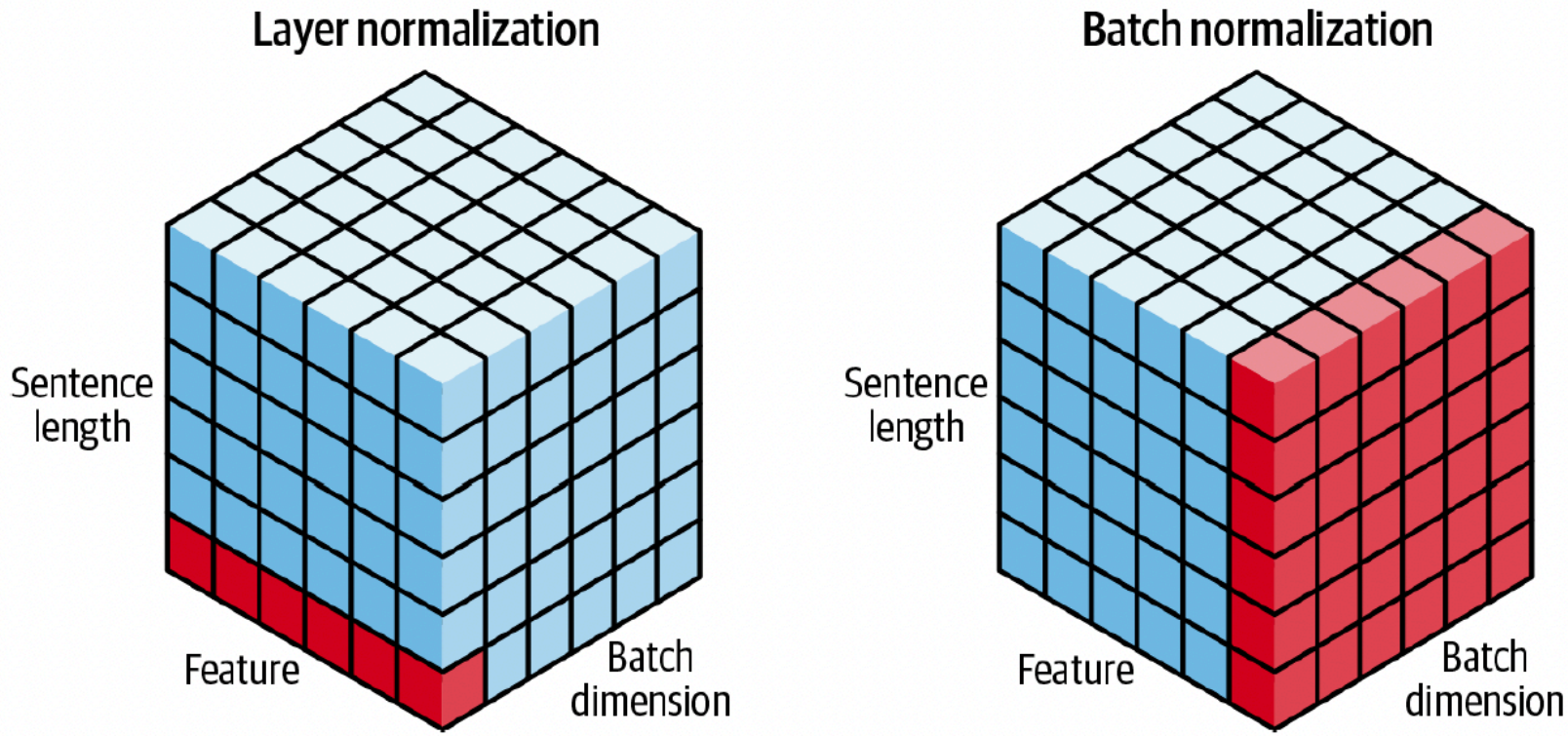
$Y = \mathrm{Concat}(Y^{(1)}, \ldots, Y^{(H)}) W_O.$

Summary

$$s \;\to\; X \;\to\; (Q, K, V) \;\to\; S = \frac{QK^\top}{\sqrt{d_k}} \;\to\; A = \mathrm{softmax}(S{+}M) \;\to\; Y = AV.$$

Layer Normalization

LayerNorm normalizes **each token's feature vector** across its **hidden dimensions**, then applies a learned scale and shift. It is used heavily in Transformers to stabilize training and improve gradient flow.



Assume the hidden representation for token t is a vector:

$$x_t \in \mathbb{R}^d.$$

LayerNorm is applied **independently for each token** (no batch statistics needed).

Compute mean and variance across features

Mean:

$$\mu_t = \frac{1}{d} \sum_{i=1}^{d} x_{t,i}.$$

Variance:

$$\sigma_t^2 = \frac{1}{d}\sum_{i=1}^{d}(x_{t,i} - \mu_t)^2.$$

Normalize the vector

$$\hat{x}_{t,i} = \frac{x_{t,i} - \mu_t}{\sqrt{\sigma_t^2 + \epsilon}}, \quad i = 1, \ldots, d.$$

Here ϵ is a small constant (e.g., 10^{-5}) to avoid division by zero.

Apply learned scale and shift

LayerNorm then applies learnable parameters:

$$\gamma \in \mathbb{R}^d, \quad \beta \in \mathbb{R}^d,$$

so the final output is:

$$\mathrm{LN}(x_t)_i = \gamma_i \hat{x}_{t,i} + \beta_i.$$

This keeps the representation normalized **but still flexible**, because the model can learn to re-scale or shift each feature dimension.

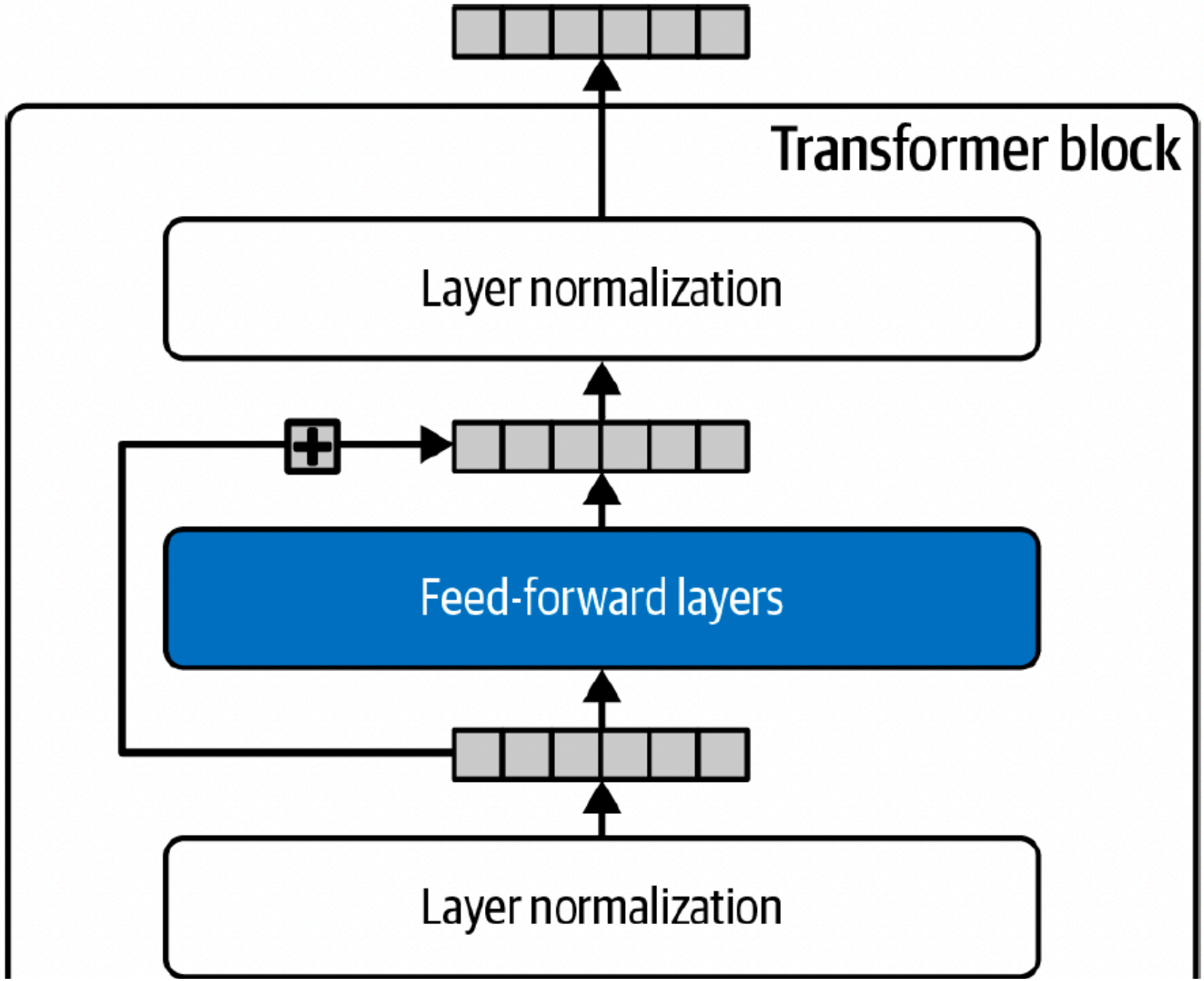
Why LayerNorm is used in Transformers

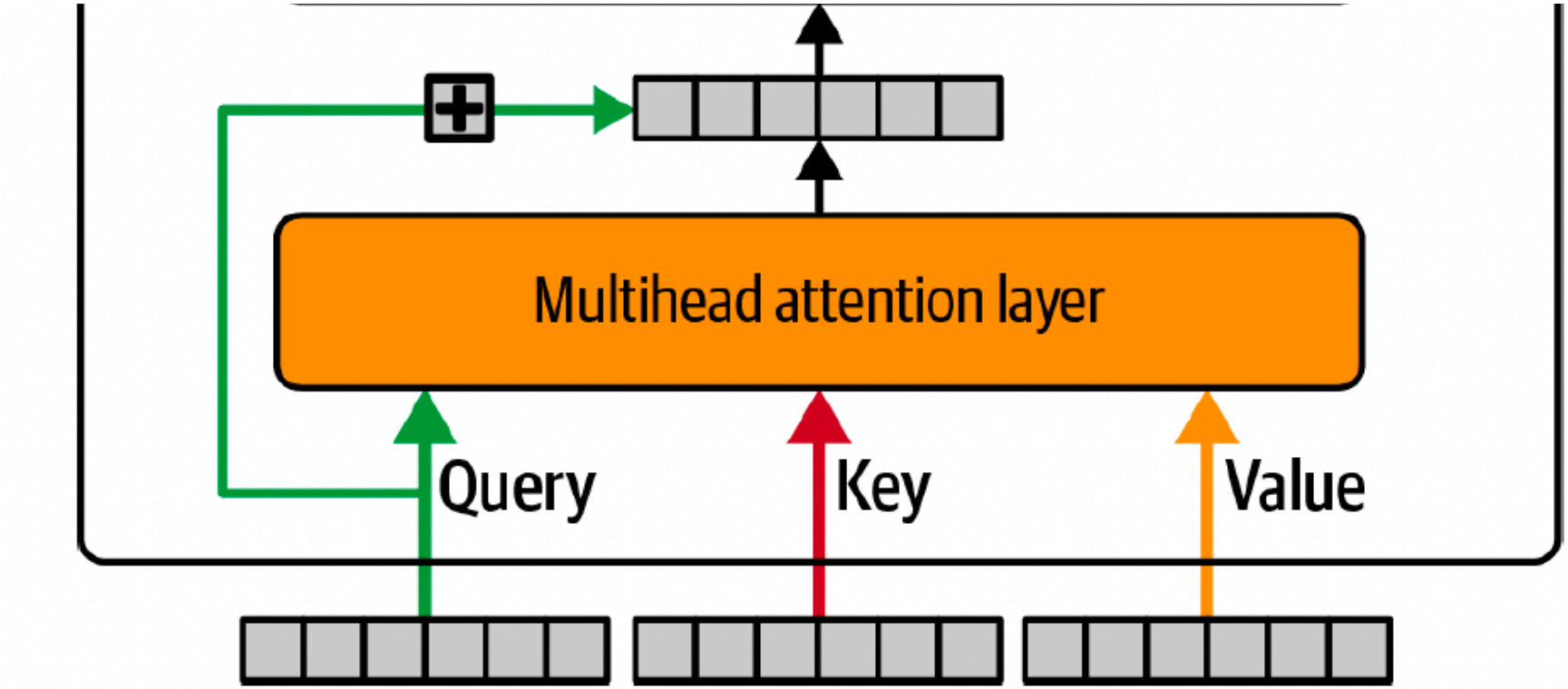
1. **Stabilizes activations across depth**
   Transformers add residuals and apply attention/MLPs repeatedly. LN prevents hidden vectors from drifting to very large or very small scales.

2. **Improves optimization and gradient flow**
   Normalized inputs make the attention and MLP sublayers easier to train, especially for deep stacks.

3. **Works well for autoregressive generation and small batches**
   Unlike BatchNorm, LayerNorm does not depend on batch statistics, so it behaves consistently even when generating one token at a time.

The Transformer Block


Transformer block
Layer normalization
Feed-forward layers
Layer normalization



Source:

Overall input/output of the block

- **Input:** a matrix of token representations
  $X \in \mathbb{R}^{T \times d}$ where T is the number of tokens and d is the hidden size. Row x_t is token t's vector.
- **Output:** a new matrix
  $X_{\text{out}} \in \mathbb{R}^{T \times d}$ same shape as X, but each token vector is more contextual.

Multi-head attention layer (orange)

Input

- **Input:** the current sequence representations $X \in \mathbb{R}^{T \times d}$.

What it produces internally

It creates three derived sequences:

- **Queries:** $Q \in \mathbb{R}^{T \times d_k}$
- **Keys:** $K \in \mathbb{R}^{T \times d_k}$
- **Values:** $V \in \mathbb{R}^{T \times d_v}$

These come from learned linear projections of X: $Q = XW_Q, \quad K = XW_K, \quad V = XW_V$.

Output

- **Output:** a new sequence
  $Y \in \mathbb{R}^{T \times d}$ (after combining heads and a final projection). Each row y_t is a weighted mixture of value vectors from other tokens.

Why it is needed

- Lets each token **look at other tokens** and pull in needed information.
- Creates short paths for long-range dependencies (token t can directly use token j).
- Multiple heads let the model learn different relationships in parallel (syntax, coreference, etc.).

Residual (skip) connection around attention (green loop + plus box)

Input

- The original sequence X
- The attention output $Y = \mathrm{MHA}(X)$

Output

- The sum (same shape): $X + Y \in \mathbb{R}^{T \times d}$.

Why it is needed

- Preserves useful information from X.
- Stabilizes training and improves gradient flow.

- Lets the attention sublayer learn refinements instead of rewriting everything.

Layer normalization after attention (first LN box)

Input

- The post-residual result: $X + \mathrm{MHA}(X)$.

Output

- A normalized sequence: $X_1 = \mathrm{LN}(X + \mathrm{MHA}(X)) \in \mathbb{R}^{T \times d}$.

Why it is needed

- Stabilizes activation scale across layers.
- Makes optimization more reliable.
- Works well even for small batches and autoregressive generation.

Feed-forward layers (blue block, FFN / MLP)

Input

- The normalized sequence: $X_1 \in \mathbb{R}^{T \times d}$.

Output

- A transformed sequence: $F = \mathrm{FFN}(X_1) \in \mathbb{R}^{T \times d}$,

where the same MLP is applied **independently to each token**.

A common FFN form is: $\mathrm{FFN}(x) = W_2\, \sigma(W_1 x + b_1) + b_2$.

Why it is needed

- Attention mainly mixes information across tokens (weighted sums).
- FFN adds **nonlinearity** and **feature creation** per token.

- Intuition: attention chooses what to read; FFN computes with what was read.

Residual (skip) connection around the FFN (black loop + plus box)

Input

- The FFN input X_1
- The FFN output $\mathrm{FFN}(X_1)$

Output

- The sum: $X_1 + \mathrm{FFN}(X_1) \in \mathbb{R}^{T \times d}$.

Why it is needed

- Same reason as the attention residual: stable deep training and preserving information.

Layer normalization after FFN (top LN box)

Input

- The post-residual FFN result: $X_1 + \mathrm{FFN}(X_1)$.

Output

- The final block output: $X_{\text{out}} = \mathrm{LN}(X_1 + \mathrm{FFN}(X_1)) \in \mathbb{R}^{T \times d}$.

Why it is needed

- Keeps the output well-scaled before the next block.
- Prevents activation drift as many blocks are stacked.

Summary of the whole block (matching the diagram)

- **Attention sublayer:** $X_1 = \mathrm{LN}(X + \mathrm{MHA}(X))$.

- **FFN sublayer:** $X_{\text{out}} = \text{LN}(X_1 + \text{FFN}(X_1))$.

Simple Implementation of a Transformer Block

Now we implement a full-fledged implementation of a **Transformer**. We only require **PyTorch**. Matplotlib is optional (used for a small visualization).

In [3]:
```
!pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
```

```
Looking in indexes: https://download.pytorch.org/whl/cu118
Requirement already satisfied: torch in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (2.5.1)
Requirement already satisfied: torchvision in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (0.20.
1)
Requirement already satisfied: filelock in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (from tor
ch) (3.17.0)
Requirement already satisfied: typing-extensions>=4.8.0 in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-pa
ckages (from torch) (4.12.2)
Requirement already satisfied: networkx in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (from tor
ch) (3.4.2)
Requirement already satisfied: jinja2 in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (from torc
h) (3.1.6)
Requirement already satisfied: fsspec in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (from torc
h) (2025.7.0)
Requirement already satisfied: sympy!=1.13.2,>=1.13.1 in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-pack
ages (from torch) (1.13.3)
Requirement already satisfied: numpy in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (from torchv
ision) (1.26.4)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packa
ges (from torchvision) (11.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages
(from sympy!=1.13.2,>=1.13.1->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/albareeq/miniconda3/envs/ml/lib/python3.11/site-packages (f
rom jinja2->torch) (3.0.2)
```

In [19]:
```
import sys, platform
print(sys.version)
print(platform.platform())
```

```
try:
    import torch
    print("torch:", torch.__version__)
    print("cuda:", torch.cuda.is_available())
except Exception as e:
    print("torch import failed:", e)
```

```
3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0]
Linux-6.8.0-63-generic-x86_64-with-glibc2.39
torch: 2.8.0+cu128
cuda: True
```

In [20]:

```
# If you need installs (uncomment):

import math
import time
import random
import urllib.request
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional

import torch
import torch.nn as nn
import torch.nn.functional as F

try:
    import matplotlib.pyplot as plt
    HAVE_PLT = True
except Exception:
    HAVE_PLT = False

# Reproducibility
seed = 1337
random.seed(seed)
torch.manual_seed(seed)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("torch:", torch.__version__)
print("device:", device)
```

```
torch: 2.8.0+cu128
device: cuda
```

Download a simple text corpus

We will use **Tiny Shakespeare** (popular small LM demo corpus). If download fails, we fall back to a short built-in text. We will train the model to **predict** words based on this **corpus**.

In [21]:
```python
def download_text(url: str, timeout: int = 15) -> Optional[str]:
    try:
        with urllib.request.urlopen(url, timeout=timeout) as r:
            return r.read().decode("utf-8", errors="replace")
    except Exception as e:
        print("[download] failed:", e)
        return None

url = "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt"
text = download_text(url)

if text is None:
    text = (
        "To be, or not to be, that is the question:\n"
        "Whether 'tis nobler in the mind to suffer\n"
        "The slings and arrows of outrageous fortune...\n"
    )
    print("[data] using fallback mini-corpus")

print("chars:", len(text))
print("preview:\n", text[:600])
```

```
chars: 1115394
preview:
 First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.

All:
We know't, we know't.

First Citizen:
Let us kill him, and we'll have corn at our own price.
Is't a verdict?

All:
No more talking on't; let it be done: away, away!

Second Citizen:
One word, good citizens.

First Citizen:
We are accounted poor citizens, the patricians good.
What authority surfeits on would relieve us: if they
would yield us
```

Byte-Pair Encoding (BPE) Tokenization

Modern language models do not operate directly on raw characters or raw words. Instead, they convert text into **tokens** drawn from a fixed vocabulary.

BPE (Byte-Pair Encoding) is a widely used tokenization method that builds a vocabulary of **subword units**.
It provides a practical balance between:

- character-level modeling (very flexible, but long sequences)
- word-level modeling (short sequences, but many unknown words)

The problem BPE solves

If we tokenize by **words**, we get:

- a huge vocabulary (millions of words)
- an out-of-vocabulary problem (unknown words, spelling variants, names)

If we tokenize by **characters**, we get:

- a tiny vocabulary (letters + punctuation)
- but sequences become much longer (harder and slower to model)

BPE gives a middle ground:

- **frequent patterns** become single tokens
- **rare words** are split into smaller pieces that are still in-vocabulary

Core idea of BPE

BPE starts with a vocabulary of small symbols (often characters, or bytes) and repeatedly:

1. counts all adjacent symbol pairs in the training data
2. merges the most frequent pair into a new symbol
3. repeats until a target vocabulary size is reached

This creates tokens like:

- `"th"`, `"the"`, `"ing"`, `"tion"`, `"elephant"`, etc.

So the tokenizer learns common substrings automatically.

The BPE merge algorithm

Let the initial symbol sequence for a word be:

$$w = (c_1, c_2, \ldots, c_n),$$

where each c_i is a base symbol (character or byte).

1) Count adjacent pairs

Count occurrences of all adjacent pairs:

$$(c_1, c_2), (c_2, c_3), \ldots, (c_{n-1}, c_n).$$

Across the whole corpus, find the most frequent pair:

$$(a, b) = \arg\max_{(u,v)} \text{count}(u, v).$$

2) Merge the most frequent pair

Create a new symbol:

$$ab,$$

and replace every occurrence of a, b with ab.

3) Repeat

Repeat count-and-merge until the vocabulary size reaches a chosen limit.

Simple example

Suppose our dataset contains these words (shown as characters with an end marker `</w>`):

- `low</w>`
- `lower</w>`
- `newest</w>`
- `widest</w>`

Start with character tokens:

- `l o w </w>`
- `l o w e r </w>`
- `n e w e s t </w>`
- `w i d e s t </w>`

If `e s` is the most frequent adjacent pair, merge:

- `e s` $\rightarrow$ `es`

Then `es t` might be merged to `est`, and so on.
Eventually, you may learn tokens like:

- `low`, `lower`, `est`, `new`, `wid`, etc.

Rare words are represented as combinations of learned subwords.

Byte-level BPE in GPT-2

Standard BPE often starts from **characters**, but GPT-2 uses **byte-level BPE**:

- The base vocabulary is all 256 possible byte values $0, \ldots, 255$.
- Text is converted to UTF-8 bytes.
- BPE merges are learned over byte sequences.

Why byte-level matters

- Any Unicode text can be represented as bytes (no unknown characters).
- It avoids complicated language-specific preprocessing.

- It guarantees that every possible string is tokenizable.

So in byte-level BPE, the initial tokens are bytes, and merges build up larger byte sequences that correspond to readable substrings.

Encoding (tokenizing) with BPE after training

Once the merge rules are learned, tokenizing new text works like this:

1. start from base symbols (characters or bytes)
2. apply the learned merges in order (highest priority merges first)
3. output the final list of token ids

So tokenization is a deterministic “apply merges” procedure.

Why BPE is a good fit for language models

BPE gives you:

- a fixed vocabulary size (controls model size and speed)
- no unknown words (rare words are decomposed)
- shorter sequences than characters
- better handling of names, spelling variations, and morphologically rich languages

Summary

BPE tokenization learns a vocabulary of frequent subword patterns by repeatedly merging common adjacent symbol pairs, giving efficient and flexible token representations for language models.

GPT-2 uses byte-level BPE:

Below is a compact, educational byte-level BPE implementation.

In [22]:
```
class ByteBPE:
    '''
```

```
    Minimal byte-level BPE:
    - Base vocab: 256 byte tokens [0..255]
    - Learn merges on training corpus to reach vocab_size
    - Encode by applying merges in learned order (slow but OK for small corpora)
    '''
    def __init__(self, vocab_size: int = 2000):
        assert vocab_size >= 256
        self.vocab_size = vocab_size
        self.merges: List[Tuple[int, int, int]] = []  # (a, b, new_id) in learned order
        self.id_to_bytes: Dict[int, bytes] = {i: bytes([i]) for i in range(256)}
        self.bytes_to_id: Dict[bytes, int] = {bytes([i]): i for i in range(256)}

    @staticmethod
    def _pair_frequencies(seqs: List[List[int]]) -> Dict[Tuple[int, int], int]:
        freqs: Dict[Tuple[int, int], int] = {}
        for s in seqs:
            for i in range(len(s) - 1):
                p = (s[i], s[i + 1])
                freqs[p] = freqs.get(p, 0) + 1
        return freqs

    @staticmethod
    def _merge_pair(seq: List[int], a: int, b: int, new_id: int) -> List[int]:
        out: List[int] = []
        i = 0
        L = len(seq)
        while i < L:
            if i < L - 1 and seq[i] == a and seq[i + 1] == b:
                out.append(new_id)
                i += 2
            else:
                out.append(seq[i])
                i += 1
        return out

    def train(self, text: str, verbose_every: int = 200) -> None:
        data = text.encode("utf-8", errors="replace")
        # Simple chunking into "documents" for pair stats
        chunk = 2000
        seqs = [list(data[i : i + chunk]) for i in range(0, len(data), chunk) if len(data[i : i + chunk]) > 2]
```

```
        next_id = 256
        target_merges = self.vocab_size - 256

        for m in range(target_merges):
            freqs = self._pair_frequencies(seqs)
            if not freqs:
                break
            (a, b), count = max(freqs.items(), key=lambda kv: kv[1])
            if count < 2:
                break

            new_id = next_id
            next_id += 1

            # Record bytes expansion for decode
            self.id_to_bytes[new_id] = self.id_to_bytes[a] + self.id_to_bytes[b]
            self.bytes_to_id[self.id_to_bytes[new_id]] = new_id

            # Apply merge to all sequences
            seqs = [self._merge_pair(s, a, b, new_id) for s in seqs]
            self.merges.append((a, b, new_id))

            if verbose_every and (m + 1) % verbose_every == 0:
                print(f"[BPE] merges learned: {m+1}/{target_merges}")

        print(f"[BPE] final vocab size: {len(self.id_to_bytes)} (target {self.vocab_size})")

    def encode(self, text: str) -> List[int]:
        seq = list(text.encode("utf-8", errors="replace"))
        for (a, b, new_id) in self.merges:
            seq = self._merge_pair(seq, a, b, new_id)
        return seq

    def decode(self, ids: List[int]) -> str:
        b = b"".join(self.id_to_bytes[i] for i in ids)
        return b.decode("utf-8", errors="replace")


def token_repr(tok_id: int, tokenizer: ByteBPE) -> str:
    bb = tokenizer.id_to_bytes[tok_id]
    s = bb.decode("utf-8", errors="replace")
```

```
    s = s.replace("\n", "\\n").replace("\t", "\\t").replace("\r", "\\r")
    if len(s) > 30:
        s = s[:27] + "..."
    return f"id={tok_id:4d} bytes={list(bb)[:10]}{'...' if len(bb)>10 else ''} text='{s}'"
```

Train BPE on the corpus

BPE must be trained on a specific corpus of text. This will allow us to determine essentually how to **tokenize** this specific **corpus**.

In [23]:
```
# we need vocabulary size because we are learning merges to reach that vocab size

vocab_size = 1200  # increase for better compression (slower training)
tokenizer = ByteBPE(vocab_size=vocab_size)

t0 = time.time()
tokenizer.train(text, verbose_every=200)
print("BPE training seconds:", round(time.time() - t0, 2))

print("Example base byte tokens:")
for tid in [ord(' '), ord('e'), ord('\n')]:
    print(" ", token_repr(tid, tokenizer))
```

```
[BPE] merges learned: 200/944
[BPE] merges learned: 400/944
[BPE] merges learned: 600/944
[BPE] merges learned: 800/944
[BPE] final vocab size: 1200 (target 1200)
BPE training seconds: 122.87
Example base byte tokens:
  id=  32 bytes=[32] text=' '
  id= 101 bytes=[101] text='e'
  id=  10 bytes=[10] text='\n'
```

BPE Encode / Decode examples

Note that the endoding encodes the input text into a number of tokens and the output is **not** the token (e.g., ROME) but an **id** (e.g., 923) of the **token**.

In [24]:
```python
sample = "ROMEO: But, soft! what light through yonder window breaks?\n"
ids = tokenizer.encode(sample)

print("Original:", repr(sample))
print("Token count:", len(ids))
print("First 30 token IDs:", ids[:30])
print("\nDecoded:", repr(tokenizer.decode(ids)))

print("\nFirst 20 tokens (expanded):")
for tid in ids[:20]:
    print("  ", token_repr(tid, tokenizer))
```

```
Original: 'ROMEO: But, soft! what light through yonder window breaks?\n'
Token count: 26
First 30 token IDs: [923, 79, 540, 66, 1107, 115, 289, 116, 33, 730, 306, 819, 257, 1007, 32, 121, 274, 580, 288, 66
8, 284, 297, 390, 107, 115, 480]

Decoded: 'ROMEO: But, soft! what light through yonder window breaks?\n'

First 20 tokens (expanded):
   id= 923 bytes=[82, 79, 77, 69] text='ROME'
   id=  79 bytes=[79] text='O'
   id= 540 bytes=[58, 32] text=': '
   id=  66 bytes=[66] text='B'
   id=1107 bytes=[117, 116, 44, 32] text='ut, '
   id= 115 bytes=[115] text='s'
   id= 289 bytes=[111, 102] text='of'
   id= 116 bytes=[116] text='t'
   id=  33 bytes=[33] text='!'
   id= 730 bytes=[32, 119, 104, 97, 116, 32] text=' what '
   id= 306 bytes=[108, 105] text='li'
   id= 819 bytes=[103, 104, 116, 32] text='ght '
   id= 257 bytes=[116, 104] text='th'
   id=1007 bytes=[114, 111, 117, 103, 104] text='rough'
   id=  32 bytes=[32] text=' '
   id= 121 bytes=[121] text='y'
   id= 274 bytes=[111, 110] text='on'
   id= 580 bytes=[100, 101, 114] text='der'
   id= 288 bytes=[32, 119] text=' w'
   id= 668 bytes=[105, 110, 100] text='ind'
```

Show how merges change the tokenization (illustration)

In [25]:
```python
def encode_with_first_n_merges(text: str, tokenizer: ByteBPE, n_merges: int) -> List[int]:
    seq = list(text.encode("utf-8", errors="replace"))
    for (a, b, new_id) in tokenizer.merges[:n_merges]:
        seq = tokenizer._merge_pair(seq, a, b, new_id)
    return seq

demo = "To be, or not to be."
print("demo text:", demo)

for n in [0, 10, 50, 150]:
    ids_n = encode_with_first_n_merges(demo, tokenizer, n)
    pieces = [tokenizer.id_to_bytes[i].decode("utf-8", errors="replace").replace("\n","\\n") for i in ids_n]
    print(f"\nFirst {n} merges -> {len(ids_n)} tokens:")
    print(pieces)
```

```
demo text: To be, or not to be.

First 0 merges -> 20 tokens:
['T', 'o', ' ', 'b', 'e', ',', ' ', 'o', 'r', ' ', 'n', 'o', 't', ' ', 't', 'o', ' ', 'b', 'e', '.']

First 10 merges -> 18 tokens:
['T', 'o', ' ', 'b', 'e', ', ', 'o', 'r', ' ', 'n', 'o', 't ', 't', 'o', ' ', 'b', 'e', '.']

First 50 merges -> 12 tokens:
['T', 'o ', 'b', 'e, ', 'or', ' ', 'no', 't ', 'to ', 'b', 'e', '.']

First 150 merges -> 10 tokens:
['T', 'o ', 'b', 'e, ', 'or', ' ', 'not ', 'to ', 'be', '.']
```

Attention: A Simple Example

Single-head attention is:

$$\mathrm{Att}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Causal masking prevents position t from attending to future positions $> t$.

In [26]:
```
torch.set_printoptions(precision=3, sci_mode=False)

T = 4
# the dimension of the model and keys/values is small for clarity; in practice these would be larger
d_model = 6
d_k = d_model  # single head for clarity

# Fixed input sequence of token embeddings (T x d_model)
X = torch.tensor([
    [ 1.0,  0.0,  0.5, -1.0,  0.0,  0.2],
    [ 0.0,  1.0, -0.5,  0.0,  1.0, -0.2],
    [ 1.0,  1.0,  0.0,  0.5, -0.5,  0.1],
    [-0.5,  0.2,  1.0,  0.0,  0.5,  1.0],
], dtype=torch.float32)  # shape: (T=4, d_model=6)

# The learned projection matrices for queries, keys, and values (d_model x d_k)
Wq = torch.randn(d_model, d_k)
Wk = torch.randn(d_model, d_k)
Wv = torch.randn(d_model, d_k)

# Compute queries, keys, values
Q = X @ Wq
K = X @ Wk
V = X @ Wv

# Compute attention scores (T x T)
scores = (Q @ K.T) / math.sqrt(d_k)

# Causal mask to prevent attending to future tokens
causal_mask = torch.tril(torch.ones(T, T)).bool()
masked_scores = scores.masked_fill(~causal_mask, float("-inf"))

# Softmax to get attention weights, then weighted sum of values
# Note: the masked scores will have -inf for disallowed positions, so softmax will give zero weight to those
# This is the core of the self-attention mechanism: each token's output is a weighted sum of the value vectors,
# where the weights are determined by the compatibility of the query with the keys, and masked to prevent looking a
# Softmax is row-wise, so each token's weights sum to 1 over the allowed positions.
```

```python
A = torch.softmax(masked_scores, dim=-1)

# Mutiply weights by values to get output (T x d_k)
Y = A @ V

print("Scores (unmasked):\n", scores)
print("\nCausal mask (True=allowed):\n", causal_mask)
print("\nScores (masked):\n", masked_scores)
print("\nAttention weights:\n", A)
print("\nOutput = A @ V:\n", Y)
```

```
Scores (unmasked):
 tensor([[-2.857, -1.356,  1.130,  0.238],
        [ 0.164,  2.260, -4.478,  1.372],
        [ 0.234,  0.585, -0.854,  0.022],
        [-3.177, -0.115, -3.591,  2.216]])

Causal mask (True=allowed):
 tensor([[ True, False, False, False],
        [ True,  True, False, False],
        [ True,  True,  True, False],
        [ True,  True,  True,  True]])

Scores (masked):
 tensor([[-2.857,   -inf,   -inf,   -inf],
        [ 0.164,  2.260,   -inf,   -inf],
        [ 0.234,  0.585, -0.854,   -inf],
        [-3.177, -0.115, -3.591,  2.216]])

Attention weights:
 tensor([[1.000, 0.000, 0.000, 0.000],
        [0.109, 0.891, 0.000, 0.000],
        [0.363, 0.515, 0.122, 0.000],
        [0.004, 0.088, 0.003, 0.905]])

Output = A @ V:
 tensor([[ 0.502,  1.637, -1.291, -0.671,  0.634,  1.495],
        [ 2.508,  0.491, -1.747,  0.617,  0.238,  1.750],
        [ 1.571,  0.788, -1.552,  0.274,  0.440,  1.271],
        [ 1.114, -0.042, -2.912, -1.495,  1.739,  0.940]])
```

In [27]:
```
!pip install matplotlib --quiet
```

In [28]:
```
import torch, math
import matplotlib.pyplot as plt

X_np = X.numpy()
Y_np = Y.detach().numpy()
D_np = (Y - X).detach().numpy()

token_labels = [f"token {i+1}" for i in range(T)]
dim_labels = [f"d{i+1}" for i in range(d_model)]

# Heatmaps with clean integer ticks
plt.figure(figsize=(12,4))

ax1 = plt.subplot(1,3,1)
im1 = ax1.imshow(X_np, aspect='auto')
ax1.set_title("Input embeddings X (T×d)")
ax1.set_xlabel("Embedding dim")
ax1.set_ylabel("Token")
ax1.set_xticks(range(d_model))
ax1.set_xticklabels(dim_labels)
ax1.set_yticks(range(T))
ax1.set_yticklabels(token_labels)
plt.colorbar(im1, ax=ax1, fraction=0.046, pad=0.04)

ax2 = plt.subplot(1,3,2)
im2 = ax2.imshow(Y_np, aspect='auto')
ax2.set_title("Attention output Y = A·V (T×d)")
ax2.set_xlabel("Embedding dim")
ax2.set_ylabel("Token")
ax2.set_xticks(range(d_model))
ax2.set_xticklabels(dim_labels)
ax2.set_yticks(range(T))
ax2.set_yticklabels(token_labels)
plt.colorbar(im2, ax=ax2, fraction=0.046, pad=0.04)

ax3 = plt.subplot(1,3,3)
im3 = ax3.imshow(D_np, aspect='auto')
ax3.set_title("Changes to Embeddings (Y - X)")
```

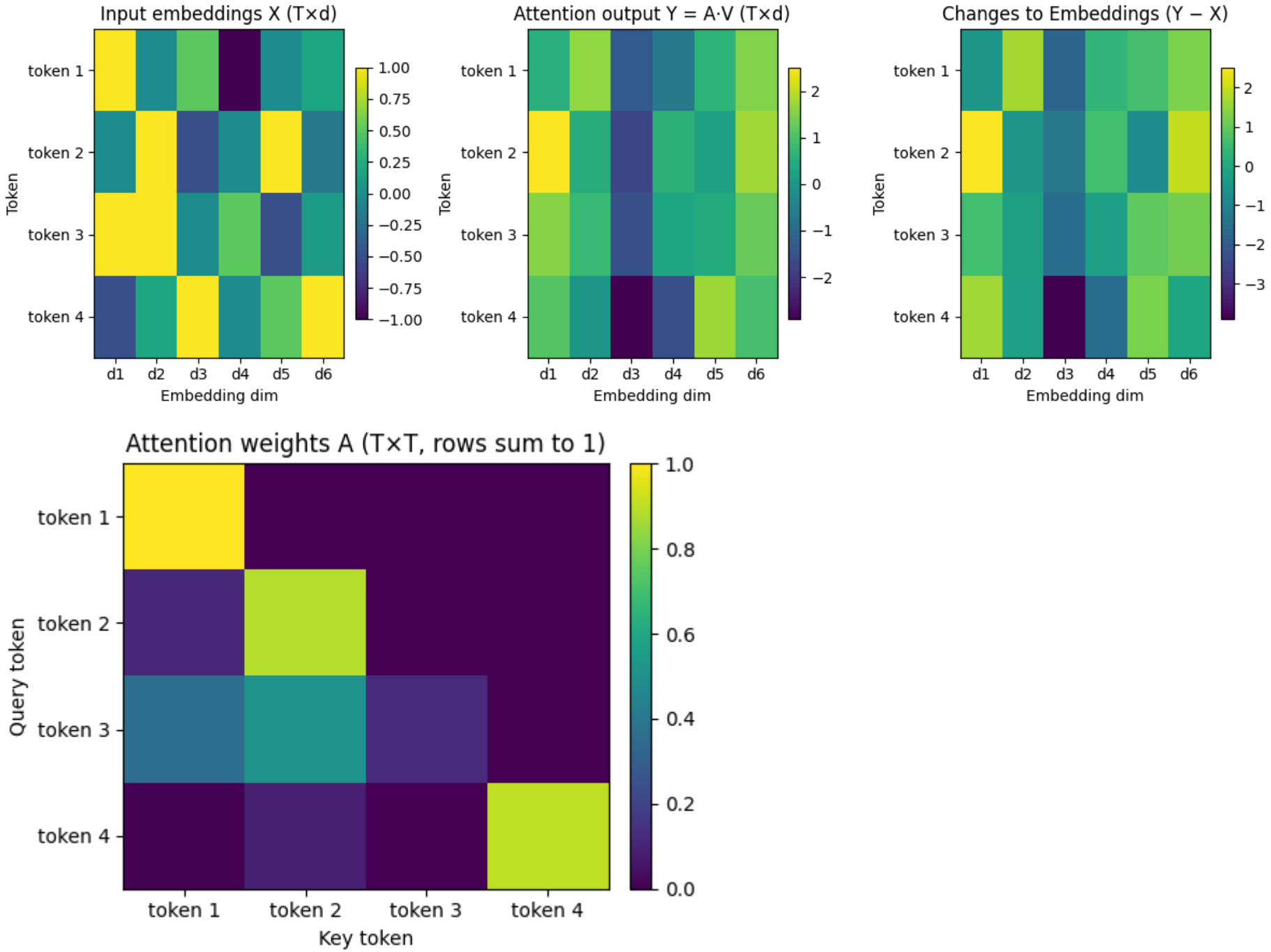
```
ax3.set_xlabel("Embedding dim")
ax3.set_ylabel("Token")
ax3.set_xticks(range(d_model))
ax3.set_xticklabels(dim_labels)
ax3.set_yticks(range(T))
ax3.set_yticklabels(token_labels)
plt.colorbar(im3, ax=ax3, fraction=0.046, pad=0.04)

plt.tight_layout()
plt.show()

plt.figure(figsize=(5.2,4))
im = plt.imshow(A.detach().numpy(), aspect='auto')
plt.title("Attention weights A (T×T, rows sum to 1)")
plt.xlabel("Key token")
plt.ylabel("Query token")
plt.colorbar(im, fraction=0.046, pad=0.04)

plt.xticks(range(T), token_labels)
plt.yticks(range(T), token_labels)
plt.tight_layout()
plt.show()


print("X shape:", tuple(X.shape))
print("A shape:", tuple(A.shape))
print("Y shape:", tuple(Y.shape))
```


Input embeddings X (T×d)
token 1
token 2
token 3
token 4
Token
d1
d2
d3
d4
d5
d6
Embedding dim
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Attention output Y = A·V (T×d)
2
1
0
−1
−2
Changes to Embeddings (Y − X)
3
Attention weights A (T×T, rows sum to 1)
Query token
Key token
0.8
0.6
0.4
0.2
0.0

```
X shape: (4, 6)
A shape: (4, 4)
Y shape: (4, 6)
```

GELU MLP in a Transformer (FFN)

Inside each Transformer block, the **MLP / Feed-Forward Network (FFN)** is a position-wise network applied **independently to each token vector**. It adds **nonlinearity** and **capacity** after attention has mixed information across tokens.

Input and output (per token)

For token t, let its vector be:

$x_t \in \mathbb{R}^d$.

The FFN outputs another vector with the **same dimension**:

$\mathrm{FFN}(x_t) \in \mathbb{R}^d$.

The standard Transformer MLP form

The most common FFN uses two linear layers with an activation in between:

$$\mathrm{FFN}(x) = W_2\,\phi(W_1 x + b_1) + b_2,$$

where:

- $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$ (expands dimension)
- $b_1 \in \mathbb{R}^{d_{\text{ff}}}$
- $W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$ (projects back)
- $b_2 \in \mathbb{R}^d$
- $\phi(\cdot)$ is typically **GELU** in GPT-style models

Typically:

$d_{\text{ff}} \approx 4d.$

So the MLP expands the representation (more capacity), applies a nonlinearity, then compresses it back.

What is GELU?

GELU stands for **Gaussian Error Linear Unit**. It behaves like a smooth, probabilistic version of ReLU.

Exact definition:

$$\text{GELU}(x) = x\,\Phi(x),$$

where $\Phi(x)$ is the CDF of a standard normal distribution:

$$\Phi(x) = \int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\,dt.$$

A commonly used approximation (fast and very accurate) is:

$$\text{GELU}(x) \approx 0.5x\left(1 + \tanh\left(\sqrt{\tfrac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right).$$

Why use GELU instead of ReLU?

Smooth gating rather than hard thresholding

- ReLU is a hard gate: it outputs 0 for all negative inputs.
- GELU smoothly down-weights negative inputs instead of killing them entirely.

So GELU behaves like:

> "keep a fraction of the signal depending on how positive it is."

Better optimization in practice for Transformers

In many Transformer LMs, GELU empirically trains better than ReLU:

- smoother gradients
- slightly better accuracy/perplexity

Why do we need an MLP at all (if we already have attention)?

Attention mostly does **weighted averaging** of value vectors:

$$y_t = \sum_j \alpha_{t,j} v_j,$$

which is primarily a mixing operation.

The FFN/MLP then:

- creates new nonlinear features from that mixed representation
- increases model capacity per token
- helps the model compute transformations like "is this token part of a noun phrase?" or "is this context indicating size?"

A good mental model:

- **Attention:** "What information should I read from other tokens?"
- **MLP (with GELU):** "How should I transform and combine that information into useful features?"

Matrix form over the whole sequence

If $X \in \mathbb{R}^{T \times d}$ is the sequence matrix:

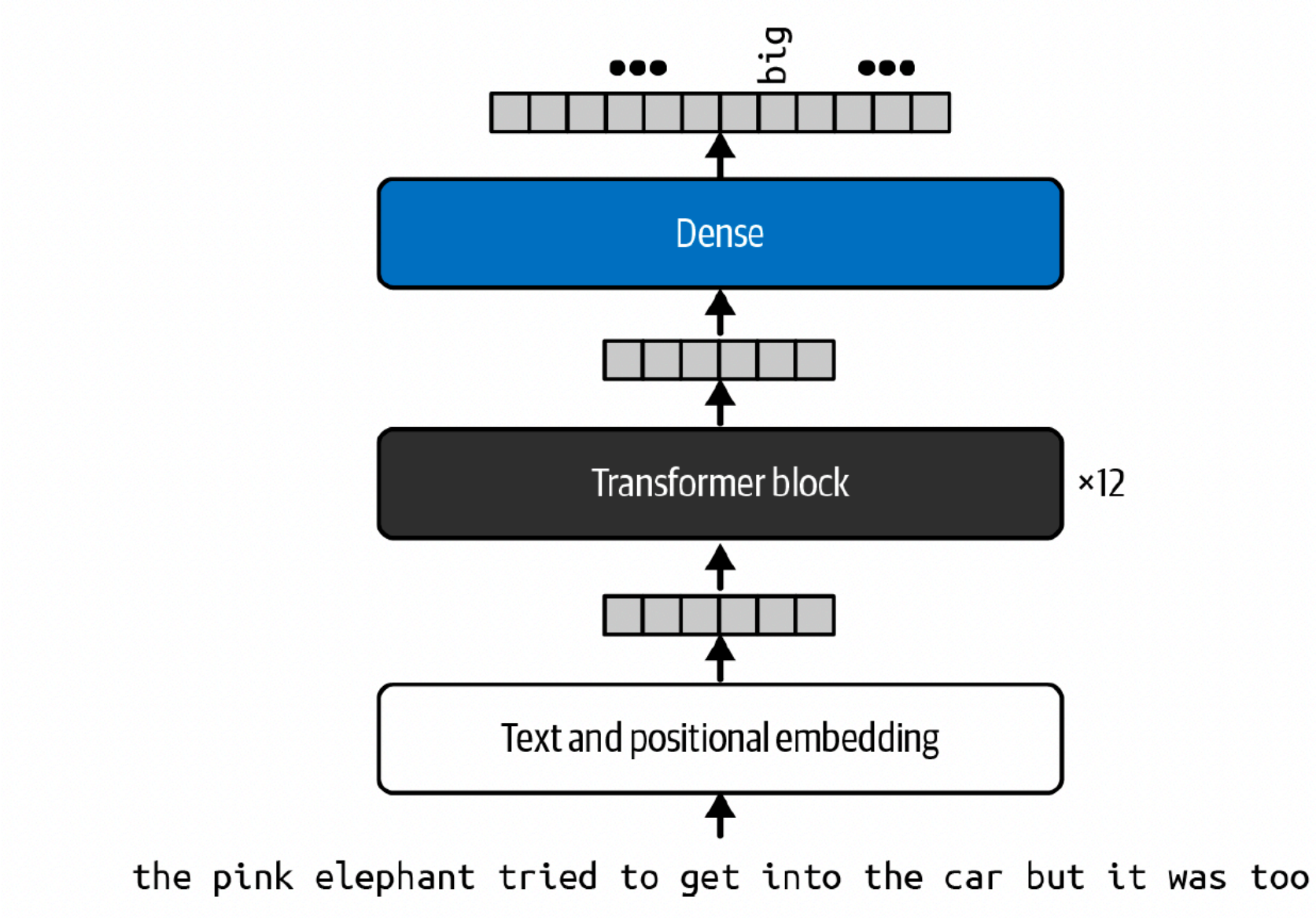
$$\mathrm{FFN}(X) = \mathrm{GELU}(X W_1^\top + \mathbf{1} b_1^\top)\, W_2^\top + \mathbf{1} b_2^\top,$$

where $\mathbf{1} \in \mathbb{R}^T$ broadcasts biases across tokens. Broadcasting means that it **repeats the same bias vector for each token**.

Summary

The Transformer GELU-MLP is a **two-layer, per-token network** that expands the hidden dimension, applies **GELU** for smooth nonlinear gating, then projects back—adding crucial nonlinear compute beyond attention's mixing.

How training works with GPT-style models



A GPT-style language model is trained to do **next-token prediction**.

Suppose you have a tokenized sequence:

$$s_1, s_2, \ldots, s_T, \quad s_t \in \{1, \ldots, V\}.$$

Build training pairs by shifting by one

We form:

- **Input** sequence x: $x = [s_1, s_2, \ldots, s_{T-1}]$
- **Target** sequence y (shifted left by one): $y = [s_2, s_3, \ldots, s_T]$

So each position t in x is asked to predict the **next** token:

$$\text{predict } y_t = s_{t+1} \text{ from } x_{\le t} = [s_1, \ldots, s_t].$$

In code, this is often done with contiguous blocks:

- $x = \text{tokens}[i : i + T]$
- $y = \text{tokens}[i + 1 : i + T + 1]$

The model produces logits for every position

Given x, the Transformer outputs a hidden state for each position, then maps it to vocabulary logits:

$$H = \text{Transformer}(x) \in \mathbb{R}^{(T-1)\times d}.$$

Final linear "LM head":

$$Z = HW_{\text{out}} + b \in \mathbb{R}^{(T-1)\times V}.$$

- Row $Z_t \in \mathbb{R}^V$ contains the logits for predicting y_t.

Convert logits to probabilities with softmax:

$$p_\theta(\cdot \mid x_{\le t}) = \text{softmax}(Z_t).$$

Cross-entropy compares predicted distribution to the true next token

Each target y_t is a single correct class index in $\{1, \ldots, V\}$.
Cross-entropy for one position t is:

$$\ell_t = -\log p_\theta(y_t \mid x_{\le t}).$$

Since:

$$p_\theta(y_t \mid x_{\le t}) = \frac{\exp(Z_{t,y_t})}{\sum_{v=1}^{V} \exp(Z_{t,v})},$$

the loss can be written as:

$$\ell_t = -\, Z_{t,y_t} + \log\left(\sum_{v=1}^{V} \exp(Z_{t,v})\right).$$

Total loss is summed (or averaged) over all positions (and batch)

For a single sequence:

$$\mathcal{L}(\theta) = \sum_{t=1}^{T-1} \ell_t = -\sum_{t=1}^{T-1} \log p_\theta(y_t \mid x_{\le t}).$$

For a batch of sequences, we sum/average across batch and time. This corresponds to the autoregressive maximum likelihood objective:

$$p_\theta(s_{1:T}) = \prod_{t=1}^{T} p_\theta(s_t \mid s_{<t}),$$

trained by minimizing negative log-likelihood.

What backprop updates

The cross-entropy loss provides gradients that update:

- the final output projection W_{out}, b (so logits become accurate)
- all Transformer block parameters (attention + MLP + LayerNorm)
- token and positional embeddings

So the model learns internal representations that make the correct next token high-probability at each position.

A small GPT-2 style Decoder-only Transformer

This impements:

- Token embeddings W_e and learned positional embeddings W_p
- Pre-LN Transformer blocks with **masked multi-head self-attention**
- GELU MLP
- Weight tying between embedding and LM head

In [29]:
```python
# vocab_size: size of the token vocabulary
# block_size: maximum context length (sequence length)
# n_layer: number of transformer blocks
# n_head: number of attention heads
# n_embd: dimensionality of token embeddings and model hidden states

@dataclass
class GPTConfig:
    vocab_size: int
    block_size: int = 128
    n_layer: int = 4
    n_head: int = 4
    n_embd: int = 256
    dropout: float = 0.1

# This is a minimal implementation of the core components of a GPT-like transformer model,
# focusing on the causal self-attention mechanism and the MLP feedforward network.
# The Block class combines these with layer normalization and residual connections,
```

```python
# and the GPT class puts everything together with token and positional embeddings,
# and a final linear layer for language modeling.

class CausalSelfAttention(nn.Module):
    def __init__(self, cfg: GPTConfig):
        super().__init__()
        assert cfg.n_embd % cfg.n_head == 0
        self.cfg = cfg
        # Each head has dimension head_dim = n_embd // n_head
        self.head_dim = cfg.n_embd // cfg.n_head

        # In practice, we would have separate projection matrices for each head, but
        # for simplicity we use one big matrix and split it later.

        # it is 3 * n_embd because we compute q, k, v in one go for efficiency
        # the qkv linear layer will take input of shape (B, T, n_embd) and output (B, T, 3 * n_embd)
        # each of q, k, v will then be of shape (B, T, n_embd) after splitting and have their own
        # bias terms. This is a common optimization to compute all three in one matrix multiply.

        self.qkv = nn.Linear(cfg.n_embd, 3 * cfg.n_embd, bias=True)
        self.proj = nn.Linear(cfg.n_embd, cfg.n_embd, bias=True)

        # Dropout for attention weights and output projection
        self.attn_drop = nn.Dropout(cfg.dropout)
        self.resid_drop = nn.Dropout(cfg.dropout)

        # mask to prevent attention to future tokens (shape: 1, 1, block_size, block_size)
        mask = torch.tril(torch.ones(cfg.block_size, cfg.block_size)).view(1, 1, cfg.block_size, cfg.block_size)

        # register as buffer so it gets moved to the correct device with the model and is not a parameter
        # register_buffer is used for tensors that are not learnable parameters but should be part of the model
        # state (e.g. moved to GPU with the model, saved in checkpoints)

        self.register_buffer("causal_mask", mask)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x has shape (B, T, C) where B=batch size, T=sequence length, C=n_embd
        B, T, C = x.shape

        # do the qkv projection in one go, then split into q, k, v
        qkv = self.qkv(x)
```

```
        # now we have the big qkv tensor of shape (B, T, 3 * n_embd), we split it into q, k, v
        # each of shape (B, T, n_embd), dim=2 because the last dimension is the one we want to split
        # last dimension of qkv is 3 * n_embd, we split it into 3 parts of size n_embd each for q, k, v

        q, k, v = qkv.split(C, dim=2)

        # now we have q, k, v each of shape (B, T, n_embd), we need to reshape them to separate the heads
        # we want to reshape to (B, T, n_head, head_dim) and then transpose to (B, n_head, T, head_dim)
        # for attention computation. This allows us to compute attention for all heads in parallel using
        # batch matrix multiplication. transpose(1, 2) moves the T dimension to the end so we can do
        # batch matmul with k and v

        q = q.view(B, T, self.cfg.n_head, self.head_dim).transpose(1, 2)
        k = k.view(B, T, self.cfg.n_head, self.head_dim).transpose(1, 2)
        v = v.view(B, T, self.cfg.n_head, self.head_dim).transpose(1, 2)

        # calculate attention scores (B, n_head, T, T) by batch matrix multiplying q and k^T,
        # then scale by sqrt(head_dim). transpose(-2, -1) on k swaps the last two dimensions so
        # we get (B, n_head, head_dim, T) which allows us to do batch matmul with q to get
        # (B, n_head, T, T) attention scores for all heads in parallel.

        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)

        # apply causal mask to prevent attending to future tokens; masked positions get -inf
        # so that softmax gives them zero weight
        att = att.masked_fill(self.causal_mask[:, :, :T, :T] == 0, float("-inf"))

        # do the softmax to get attention weights, then apply dropout
        # this is done row-wise so that each query token's weights sum to 1 over the allowed key tokens
        # this is why dim=-1 is used in softmax, to apply it to the last dimension (the key token dimension)

        w = torch.softmax(att, dim=-1)
        w = self.attn_drop(w)

        # perform the weighted sum of the values; w has shape (B, n_head, T, T) and v has
        # shape (B, n_head, T, head_dim) the result y will have shape (B, n_head, T, head_dim)
        y = w @ v

        # reshape y back to (B, T, n_embd) by first transposing to (B, T, n_head, head_dim) and then
        # merging the last two dimensions. This combines the outputs of all heads back into a single
```

```
        # vector for each token. The final linear projection then mixes the information from the heads

        y = y.transpose(1, 2).contiguous().view(B, T, C)

        # output projection and dropout. resid_drop is applied to the output of the attention before
        # adding it back to the input (residual connection)because in the full transformer block
        # we have x = x + attn(x) and we want to apply dropout to the attn output before adding it to x.

        y = self.resid_drop(self.proj(y))

        return y


# The MLP (feedforward) part of the transformer block, which consists of two linear layers
# with a nonlinearity in between.

class MLP(nn.Module):
    def __init__(self, cfg: GPTConfig):
        super().__init__()
        # The first linear layer expands the dimensionality from n_embd to 4 * n_embd, which is a
        # common choice for the hidden layer size in the MLP.
        self.fc = nn.Linear(cfg.n_embd, 4 * cfg.n_embd)
        # The second linear layer projects back down to n_embd. This allows the MLP to mix information across
        # the embedding dimensions and add nonlinearity, which is important for the model's expressiveness.
        self.proj = nn.Linear(4 * cfg.n_embd, cfg.n_embd)
        # Dropout is applied to the output of the MLP before adding it back to the input (residual connection)
        self.drop = nn.Dropout(cfg.dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.fc(x)
        x = F.gelu(x)
        x = self.proj(x)
        x = self.drop(x)
        return x

# A single transformer block, which consists of a layer norm, followed by causal self-attention,
# followed by another layer norm and an MLP feedforward network, with residual connections around each part.

class Block(nn.Module):
    def __init__(self, cfg: GPTConfig):
        super().__init__()
```

```
        # Layer normalization before attention and MLP is a common design choice in transformer blocks,
        # often referred to as "pre-norm". This helps with training stability. The residual connections
        # are added after the attention and MLP, so the input to each is normalized before being processed.
        self.ln1 = nn.LayerNorm(cfg.n_embd)

        # The causal self-attention module computes the attention output for the block.
        # It takes the normalized input, computes the attention, and produces an output of the same
        # shape (B, T, n_embd).

        self.attn = CausalSelfAttention(cfg)

        # Another layer norm before the MLP, and then the MLP itself. The MLP takes the output of the attention,
        # applies a feedforward transformation, and produces an output of the same shape
        # (B, T, n_embd).

        self.ln2 = nn.LayerNorm(cfg.n_embd)

        # The MLP is applied after the attention output and has its own residual connection. The final output of th
        # is the input plus the attention output plus the MLP output, with layer norms applied before each.
        # This allows the block to learn both local interactions (through attention) and more
        # complex transformations
        self.mlp = MLP(cfg)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # The input x is first normalized and passed through the attention module,
        # and the result is added back to x (residual connection).
        x = x + self.attn(self.ln1(x))
        # Then the result is normalized again and passed through the MLP, and that result
        # is also added back to x.
        x = x + self.mlp(self.ln2(x))
        return x


class GPT(nn.Module):
    def __init__(self, cfg: GPTConfig):
        super().__init__()
        self.cfg = cfg

        # Token embedding layer maps token IDs to dense vectors of size n_embd.
        # The input to the model is a sequence of token IDs, and the embedding layer converts
```

```
        # these to vectors that can be processed by the transformer blocks.
        # wte stands for "word token embedding" and wpe stands for "word position embedding".
        # Embeddings are a crucial part of the model as they allow it to learn a continuous representation
        # of discrete tokens, which is essential for the model to learn patterns in the data.

        self.wte = nn.Embedding(cfg.vocab_size, cfg.n_embd)
        self.wpe = nn.Embedding(cfg.block_size, cfg.n_embd)
        self.drop = nn.Dropout(cfg.dropout)

        # The transformer blocks are created in a list, and each block is an instance of the Block
        # class defined above. The number of blocks is determined by cfg.n_layer.
        # Each block will process the output of the previous block, allowing the model to learn hierarchical
        # representations of the input sequence.

        self.blocks = nn.ModuleList([Block(cfg) for _ in range(cfg.n_layer)])

        # Final layer norm before the output projection. This is a common design choice in transformer models,
        # to normalize the output of the last block before projecting to the vocabulary size for language modeling
        self.ln_f = nn.LayerNorm(cfg.n_embd)

        # The final linear layer (lm_head) projects the output of the last transformer block to the size
        # of the vocabulary, which allows the model to produce logits for each token in the vocabulary
        # at each position in the sequence. The bias is set to False because we will tie the weights of
        # this layer to the token embedding layer,
        self.lm_head = nn.Linear(cfg.n_embd, cfg.vocab_size, bias=False)

        # weight tying: the weights of the output projection (lm_head) are tied to the token embedding
        # weights (wte) meaning they are the same matrix. This is a common technique in language models
        # that reduces the number of parameters and can improve performance.
        self.lm_head.weight = self.wte.weight  # weight tying

        # Initialize weights using the _init_weights method defined below, which applies a normal
        # distribution to the weights of linear and embedding layers, and zeros to the biases.
        # This is important for training stability
        self.apply(self._init_weights)

        # Educational residual scaling idea to make it easier to see the effect of each block in the
        # early stages of training.
        with torch.no_grad():
            scale = 1.0 / math.sqrt(2 * cfg.n_layer)
            for b in self.blocks:
```

```
            b.attn.proj.weight.mul_(scale)
            b.mlp.proj.weight.mul_(scale)

    @staticmethod
    def _init_weights(module: nn.Module) -> None:
        if isinstance(module, (nn.Linear, nn.Embedding)):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if isinstance(module, nn.Linear) and module.bias is not None:
            nn.init.zeros_(module.bias)

    def forward(self, idx: torch.Tensor, targets: Optional[torch.Tensor] = None):

        # idx has shape (B, T) where B=batch size and T=sequence length; each element is a token ID
        B, T = idx.shape

        # check that the sequence length T does not exceed the model's block size,
        # which is the maximum context length
        assert T <= self.cfg.block_size

        # create position indices for the sequence (shape: 1, T) and get token and position embeddings
        pos = torch.arange(0, T, device=idx.device).unsqueeze(0)

        # the input to the transformer blocks is the sum of the token embeddings and the position embeddings,
        # which allows the model to learn both the identity of the tokens and their positions in the sequence.
        x = self.wte(idx) + self.wpe(pos)
        x = self.drop(x)

        # pass the input through each of the transformer blocks in sequence
        for b in self.blocks:
            x = b(x)
        x = self.ln_f(x)

        # project the output of the last block to the vocabulary size to get logits for language modeling
        logits = self.lm_head(x)

        # loss is set to None if targets is not provided, otherwise compute cross-entropy loss between
        # the logits and the targets
        loss = None
        if targets is not None:
            # logits has shape (B, T, vocab_size) and targets has shape (B, T), we need to reshape them to
            # (B*T, vocab_size) and (B*T) respectively for cross_entropy which expects 2D input for logits
```

```
            # and 1D input for targets
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))

        # the model returns the logits for each token in the sequence, and optionally the loss
        # if targets were provided
        return logits, loss

    # generate method for autoregressive generation of text. It takes an initial sequence of token IDs (idx)
    # and generates new tokens up to max_new_tokens by repeatedly feeding the current sequence through the
    # model and sampling from the output distribution to get the next token ID, which is then appended

    @torch.no_grad()
    def generate(self, idx: torch.Tensor, max_new_tokens: int, top_k: int = 50, temperature: float = 1.0) -> torch.
        self.eval()
        for _ in range(max_new_tokens):
            # For each step of generation, we take the last block_size tokens from the current sequence
            # (idx) as input to the model.
            idx_cond = idx[:, -self.cfg.block_size :]

            # We get the logits for the next token by passing idx_cond through the model.
            # The logits will have shape (B, T, vocab_size),
            # where T is the length of idx_cond. We take the logits for the last token
            logits, _ = self(idx_cond)

            # We focus on the logits for the last token in the sequence (the most recent token) to predict
            # the next token. The temperature parameter is used to control the randomness of the sampling process;
            # higher temperature leads to more random sampling.

            logits = logits[:, -1, :] / max(temperature, 1e-8)

            # top_k sampling: we keep only the top_k tokens with the highest logits and set the rest to -inf so tha
            # they are not sampled. This is a common technique to improve the quality of generated text
            # by limiting the sampling to a smaller set of likely tokens, which can help avoid generating
            # low-probability tokens that may lead to incoherent text.

            if top_k is not None and top_k > 0:
                v, _ = torch.topk(logits, k=min(top_k, logits.size(-1)))
                logits[logits < v[:, [-1]]] = float("-inf")

            # probs is the probability distribution over the next token, obtained by applying softmax
            # to the logits.
```

```
            probs = torch.softmax(logits, dim=-1)

            # We sample the next token ID from the probability distribution. torch.multinomial is used to sample
            # from the distribution defined by probs. num_samples=1 means we want to sample one token ID for
            # each example in the batch. The result will have shape (B, 1)
            next_id = torch.multinomial(probs, num_samples=1)

            # We append the sampled next token ID to the current sequence of token IDs (idx) for each example
            # in the batch.
            idx = torch.cat([idx, next_id], dim=1)
        return idx
```

Prepare tokenized dataset and batching

In [30]:

```
# Now we have defined the tokenizer and the model, we can prepare the data for training.
# We will encode the text using the ByteBPE tokenizer, split it into a training and validation set,
# and create a simple data loader that can provide batches of token sequences for training.
all_tokens = tokenizer.encode(text)
print("Total tokens:", len(all_tokens))

# We will use the first 90% of the tokens for training and the remaining 10% for validation.
n = len(all_tokens)
split = int(0.9 * n)
train_tokens = torch.tensor(all_tokens[:split], dtype=torch.long)
val_tokens   = torch.tensor(all_tokens[split:], dtype=torch.long)

# We create instances of the LMData class for the training and validation sets, which will provide
# a method to get batches of data for training.
class LMData:
    def __init__(self, tokens: torch.Tensor, block_size: int):
        self.tokens = tokens
        self.block_size = block_size

    def get_batch(self, batch_size: int, device: torch.device):
        # Randomly sample starting indices for the batch, ensuring that we have enough tokens left
        # for a full block
        ix = torch.randint(0, len(self.tokens) - self.block_size - 1, (batch_size,))

        # For each starting index, we create a sequence of input tokens (x) and target tokens (y)
```

```
        # where y is the input sequence shifted by one token to the right. This means that for
        # each position in x, the corresponding position in y is the next token that the
        # model should predict. This is the standard setup for training a language model, where t
        # he model learns to predict the next token given the previous tokens.
        # for example, if tokens are [A, B, C, D] and block_size is 3, then x would be [A, B, C]
        # and y would be [B, C, D],
        x = torch.stack([self.tokens[i : i + self.block_size] for i in ix])
        y = torch.stack([self.tokens[i + 1 : i + self.block_size + 1] for i in ix])
        return x.to(device), y.to(device)

@torch.no_grad()
def estimate_loss(model, train_data, val_data, device, iters=20, batch_size=32):
    # Set the model to evaluation mode to disable dropout and other training-specific behavior
    model.eval()
    # We will compute the average loss on both the training and validation sets by sampling a number of
    # batches
    losses = []
    for data in (train_data, val_data):
        l = []
        for _ in range(iters):
            xb, yb = data.get_batch(batch_size, device)
            _, loss = model(xb, yb)
            l.append(loss.item())
        losses.append(sum(l) / len(l))
    # After computing the losses, we set the model back to training mode so that it can continue to be
    # trained.
    model.train()
    return losses[0], losses[1]
```

```
Total tokens: 422495
```

Train the model

In [38]:

```
cfg = GPTConfig(
    vocab_size=len(tokenizer.id_to_bytes),
    block_size=128,
    n_layer=4,
    n_head=4,
    n_embd=256,
    dropout=0.1,
```

```
)
model = GPT(cfg).to(device)
print("Params (M):", sum(p.numel() for p in model.parameters()) / 1e6)

train_data = LMData(train_tokens, cfg.block_size)
val_data = LMData(val_tokens, cfg.block_size)

optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.1)

steps = 30000
batch_size = 32
eval_every = 150

t0 = time.time()
for step in range(1, steps + 1):
    xb, yb = train_data.get_batch(batch_size, device)
    _, loss = model(xb, yb)

    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    # Gradient clipping to prevent exploding gradients, which can be an issue in training deep
    # transformer models.
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    optimizer.step()

    if step % 50 == 0:
        print(f"step {step:4d}/{steps} | loss {loss.item():.4f}")

    if step % eval_every == 0:
        tr, va = estimate_loss(model, train_data, val_data, device, iters=20, batch_size=batch_size)
        print(f"[eval] step {step:4d} | train {tr:.4f} | val {va:.4f} | elapsed {time.time()-t0:.1f}s")
```

```
Params (M): 3.49952
step   50/30000 | loss 6.1906
step  100/30000 | loss 5.3867
step  150/30000 | loss 5.0271
[eval] step  150 | train 4.9497 | val 5.1642 | elapsed 1.2s
step  200/30000 | loss 4.8602
step  250/30000 | loss 4.6846
step  300/30000 | loss 4.5721
[eval] step  300 | train 4.5187 | val 4.7875 | elapsed 2.8s
step  350/30000 | loss 4.5289
step  400/30000 | loss 4.4410
step  450/30000 | loss 4.3560
[eval] step  450 | train 4.3167 | val 4.6316 | elapsed 4.7s
step  500/30000 | loss 4.2830
step  550/30000 | loss 4.2688
step  600/30000 | loss 4.2339
[eval] step  600 | train 4.1620 | val 4.5018 | elapsed 6.3s
step  650/30000 | loss 4.2024
step  700/30000 | loss 4.0787
step  750/30000 | loss 4.1167
[eval] step  750 | train 4.0247 | val 4.4031 | elapsed 8.0s
step  800/30000 | loss 4.0494
step  850/30000 | loss 4.0302
step  900/30000 | loss 4.0093
[eval] step  900 | train 3.8808 | val 4.3232 | elapsed 9.0s
step  950/30000 | loss 3.9009
step 1000/30000 | loss 3.8811
step 1050/30000 | loss 3.9280
[eval] step 1050 | train 3.7560 | val 4.2552 | elapsed 9.9s
step 1100/30000 | loss 3.7883
step 1150/30000 | loss 3.8405
step 1200/30000 | loss 3.7806
[eval] step 1200 | train 3.6613 | val 4.2232 | elapsed 10.9s
step 1250/30000 | loss 3.8050
step 1300/30000 | loss 3.6410
step 1350/30000 | loss 3.7424
[eval] step 1350 | train 3.5645 | val 4.1520 | elapsed 12.1s
step 1400/30000 | loss 3.5497
step 1450/30000 | loss 3.7230
step 1500/30000 | loss 3.5458
[eval] step 1500 | train 3.4786 | val 4.1216 | elapsed 13.2s
```

```
step 1550/30000 | loss 3.6006
step 1600/30000 | loss 3.5577
step 1650/30000 | loss 3.6048
[eval] step 1650 | train 3.4130 | val 4.0823 | elapsed 14.4s
step 1700/30000 | loss 3.5561
step 1750/30000 | loss 3.5852
step 1800/30000 | loss 3.5182
[eval] step 1800 | train 3.3235 | val 4.0324 | elapsed 15.7s
step 1850/30000 | loss 3.4296
step 1900/30000 | loss 3.4116
step 1950/30000 | loss 3.4958
[eval] step 1950 | train 3.2659 | val 4.0074 | elapsed 16.8s
step 2000/30000 | loss 3.3411
step 2050/30000 | loss 3.3821
step 2100/30000 | loss 3.3868
[eval] step 2100 | train 3.1876 | val 3.9982 | elapsed 18.5s
step 2150/30000 | loss 3.3903
step 2200/30000 | loss 3.2923
step 2250/30000 | loss 3.3158
[eval] step 2250 | train 3.1085 | val 4.0188 | elapsed 20.0s
step 2300/30000 | loss 3.2654
step 2350/30000 | loss 3.3082
step 2400/30000 | loss 3.2152
[eval] step 2400 | train 3.0592 | val 3.9862 | elapsed 21.1s
step 2450/30000 | loss 3.2182
step 2500/30000 | loss 3.3318
step 2550/30000 | loss 3.2235
[eval] step 2550 | train 3.0223 | val 3.9768 | elapsed 22.5s
step 2600/30000 | loss 3.2989
step 2650/30000 | loss 3.1712
step 2700/30000 | loss 3.1790
[eval] step 2700 | train 2.9783 | val 4.0117 | elapsed 23.9s
step 2750/30000 | loss 3.1234
step 2800/30000 | loss 3.1571
step 2850/30000 | loss 3.2260
[eval] step 2850 | train 2.9255 | val 3.9420 | elapsed 25.3s
step 2900/30000 | loss 3.0844
step 2950/30000 | loss 3.1158
step 3000/30000 | loss 3.0535
[eval] step 3000 | train 2.8405 | val 3.9814 | elapsed 26.8s
step 3050/30000 | loss 3.1043
```

```
step 3100/30000 | loss 3.1339
step 3150/30000 | loss 3.0598
[eval] step 3150 | train 2.8122 | val 3.9990 | elapsed 28.2s
step 3200/30000 | loss 3.1117
step 3250/30000 | loss 3.0130
step 3300/30000 | loss 3.0712
[eval] step 3300 | train 2.7627 | val 3.9960 | elapsed 29.8s
step 3350/30000 | loss 2.9843
step 3400/30000 | loss 2.9612
step 3450/30000 | loss 2.9704
[eval] step 3450 | train 2.6993 | val 4.0301 | elapsed 31.6s
step 3500/30000 | loss 2.9475
step 3550/30000 | loss 3.0592
step 3600/30000 | loss 3.0043
[eval] step 3600 | train 2.6671 | val 4.0156 | elapsed 32.8s
step 3650/30000 | loss 2.9690
step 3700/30000 | loss 2.9623
step 3750/30000 | loss 2.8442
[eval] step 3750 | train 2.6007 | val 4.0347 | elapsed 33.7s
step 3800/30000 | loss 2.9788
step 3850/30000 | loss 2.8657
step 3900/30000 | loss 2.8190
[eval] step 3900 | train 2.5429 | val 4.0415 | elapsed 34.7s
step 3950/30000 | loss 2.8655
step 4000/30000 | loss 2.9311
step 4050/30000 | loss 2.9093
[eval] step 4050 | train 2.5191 | val 4.0383 | elapsed 36.0s
step 4100/30000 | loss 2.7940
step 4150/30000 | loss 2.7818
step 4200/30000 | loss 2.8908
[eval] step 4200 | train 2.4675 | val 4.1076 | elapsed 37.7s
step 4250/30000 | loss 2.7979
step 4300/30000 | loss 2.8389
step 4350/30000 | loss 2.8569
[eval] step 4350 | train 2.4437 | val 4.1101 | elapsed 39.0s
step 4400/30000 | loss 2.7477
step 4450/30000 | loss 2.6500
step 4500/30000 | loss 2.7441
[eval] step 4500 | train 2.3811 | val 4.0650 | elapsed 40.2s
step 4550/30000 | loss 2.6934
step 4600/30000 | loss 2.6730
```

```
step 4650/30000 | loss 2.7368
[eval] step 4650 | train 2.3576 | val 4.1216 | elapsed 41.6s
step 4700/30000 | loss 2.7445
step 4750/30000 | loss 2.7670
step 4800/30000 | loss 2.5954
[eval] step 4800 | train 2.3006 | val 4.1181 | elapsed 43.5s
step 4850/30000 | loss 2.7958
step 4900/30000 | loss 2.6890
step 4950/30000 | loss 2.6562
[eval] step 4950 | train 2.2760 | val 4.1317 | elapsed 44.9s
step 5000/30000 | loss 2.6033
step 5050/30000 | loss 2.6952
step 5100/30000 | loss 2.6820
[eval] step 5100 | train 2.2445 | val 4.2036 | elapsed 46.3s
step 5150/30000 | loss 2.6682
step 5200/30000 | loss 2.6848
step 5250/30000 | loss 2.6001
[eval] step 5250 | train 2.1639 | val 4.1888 | elapsed 47.5s
step 5300/30000 | loss 2.6017
step 5350/30000 | loss 2.6025
step 5400/30000 | loss 2.5200
[eval] step 5400 | train 2.1569 | val 4.2445 | elapsed 49.2s
step 5450/30000 | loss 2.4980
step 5500/30000 | loss 2.5485
step 5550/30000 | loss 2.5317
[eval] step 5550 | train 2.1148 | val 4.2654 | elapsed 50.4s
step 5600/30000 | loss 2.6013
step 5650/30000 | loss 2.5152
step 5700/30000 | loss 2.4589
[eval] step 5700 | train 2.0444 | val 4.2490 | elapsed 52.1s
step 5750/30000 | loss 2.4702
step 5800/30000 | loss 2.4883
step 5850/30000 | loss 2.5477
[eval] step 5850 | train 2.0307 | val 4.2666 | elapsed 53.7s
step 5900/30000 | loss 2.4823
step 5950/30000 | loss 2.5281
step 6000/30000 | loss 2.5092
[eval] step 6000 | train 1.9731 | val 4.3226 | elapsed 55.2s
step 6050/30000 | loss 2.5385
step 6100/30000 | loss 2.4844
step 6150/30000 | loss 2.4780
```

```
[eval] step 6150 | train 1.9698 | val 4.3559 | elapsed 57.1s
step 6200/30000 | loss 2.5099
step 6250/30000 | loss 2.4555
step 6300/30000 | loss 2.4421
[eval] step 6300 | train 1.9301 | val 4.3367 | elapsed 58.5s
step 6350/30000 | loss 2.5202
step 6400/30000 | loss 2.4083
step 6450/30000 | loss 2.3582
[eval] step 6450 | train 1.9031 | val 4.3824 | elapsed 59.7s
step 6500/30000 | loss 2.3598
step 6550/30000 | loss 2.3162
step 6600/30000 | loss 2.3139
[eval] step 6600 | train 1.8382 | val 4.4069 | elapsed 61.3s
step 6650/30000 | loss 2.3561
step 6700/30000 | loss 2.2927
step 6750/30000 | loss 2.4362
[eval] step 6750 | train 1.8050 | val 4.3856 | elapsed 63.3s
step 6800/30000 | loss 2.3543
step 6850/30000 | loss 2.2983
step 6900/30000 | loss 2.3237
[eval] step 6900 | train 1.7660 | val 4.4563 | elapsed 65.3s
step 6950/30000 | loss 2.3112
step 7000/30000 | loss 2.2917
step 7050/30000 | loss 2.2455
[eval] step 7050 | train 1.7481 | val 4.4489 | elapsed 66.6s
step 7100/30000 | loss 2.2456
step 7150/30000 | loss 2.3302
step 7200/30000 | loss 2.2395
[eval] step 7200 | train 1.7157 | val 4.4332 | elapsed 68.0s
step 7250/30000 | loss 2.3700
step 7300/30000 | loss 2.2589
step 7350/30000 | loss 2.2484
[eval] step 7350 | train 1.6909 | val 4.4755 | elapsed 69.6s
step 7400/30000 | loss 2.2693
step 7450/30000 | loss 2.3005
step 7500/30000 | loss 2.2403
[eval] step 7500 | train 1.6556 | val 4.5110 | elapsed 71.1s
step 7550/30000 | loss 2.3005
step 7600/30000 | loss 2.2367
step 7650/30000 | loss 2.2255
[eval] step 7650 | train 1.6593 | val 4.5570 | elapsed 72.7s
```

```
step 7700/30000 | loss 2.3228
step 7750/30000 | loss 2.2872
step 7800/30000 | loss 2.2701
[eval] step 7800 | train 1.6121 | val 4.5596 | elapsed 74.3s
step 7850/30000 | loss 2.1671
step 7900/30000 | loss 2.2760
step 7950/30000 | loss 2.1465
[eval] step 7950 | train 1.5754 | val 4.5871 | elapsed 76.1s
step 8000/30000 | loss 2.2799
step 8050/30000 | loss 2.2565
step 8100/30000 | loss 2.3558
[eval] step 8100 | train 1.5465 | val 4.5971 | elapsed 77.9s
step 8150/30000 | loss 2.1366
step 8200/30000 | loss 2.1637
step 8250/30000 | loss 2.0670
[eval] step 8250 | train 1.5366 | val 4.6448 | elapsed 79.7s
step 8300/30000 | loss 2.1340
step 8350/30000 | loss 2.0261
step 8400/30000 | loss 2.1561
[eval] step 8400 | train 1.4995 | val 4.6347 | elapsed 81.6s
step 8450/30000 | loss 2.1359
step 8500/30000 | loss 2.1504
step 8550/30000 | loss 2.1973
[eval] step 8550 | train 1.4614 | val 4.6656 | elapsed 83.4s
step 8600/30000 | loss 2.0525
step 8650/30000 | loss 2.1248
step 8700/30000 | loss 2.1010
[eval] step 8700 | train 1.4702 | val 4.6674 | elapsed 85.2s
step 8750/30000 | loss 2.0466
step 8800/30000 | loss 2.1157
step 8850/30000 | loss 2.0270
[eval] step 8850 | train 1.4098 | val 4.7373 | elapsed 86.3s
step 8900/30000 | loss 2.1483
step 8950/30000 | loss 2.2413
step 9000/30000 | loss 2.1171
[eval] step 9000 | train 1.4067 | val 4.7085 | elapsed 87.4s
step 9050/30000 | loss 2.1434
step 9100/30000 | loss 2.0473
step 9150/30000 | loss 2.0438
[eval] step 9150 | train 1.3674 | val 4.7279 | elapsed 88.6s
step 9200/30000 | loss 2.1019
```

```
step 9250/30000 | loss 2.0607
step 9300/30000 | loss 1.8870
[eval] step 9300 | train 1.3721 | val 4.7290 | elapsed 89.6s
step 9350/30000 | loss 2.0154
step 9400/30000 | loss 1.9222
step 9450/30000 | loss 2.0512
[eval] step 9450 | train 1.3360 | val 4.7610 | elapsed 90.8s
step 9500/30000 | loss 1.9467
step 9550/30000 | loss 1.9613
step 9600/30000 | loss 2.0274
[eval] step 9600 | train 1.3220 | val 4.8080 | elapsed 91.9s
step 9650/30000 | loss 2.0358
step 9700/30000 | loss 2.0181
step 9750/30000 | loss 2.0151
[eval] step 9750 | train 1.2845 | val 4.7996 | elapsed 93.0s
step 9800/30000 | loss 1.9950
step 9850/30000 | loss 1.9813
step 9900/30000 | loss 1.9674
[eval] step 9900 | train 1.2892 | val 4.7896 | elapsed 94.1s
step 9950/30000 | loss 2.0757
step 10000/30000 | loss 2.0549
step 10050/30000 | loss 2.0370
[eval] step 10050 | train 1.2682 | val 4.8296 | elapsed 95.1s
step 10100/30000 | loss 1.8092
step 10150/30000 | loss 2.0512
step 10200/30000 | loss 2.0584
[eval] step 10200 | train 1.2249 | val 4.7891 | elapsed 96.0s
step 10250/30000 | loss 1.8870
step 10300/30000 | loss 1.9485
step 10350/30000 | loss 1.8796
[eval] step 10350 | train 1.2253 | val 4.8569 | elapsed 97.1s
step 10400/30000 | loss 1.9838
step 10450/30000 | loss 1.8811
step 10500/30000 | loss 2.0437
[eval] step 10500 | train 1.2075 | val 4.8657 | elapsed 98.3s
step 10550/30000 | loss 1.9392
step 10600/30000 | loss 1.8649
step 10650/30000 | loss 1.9272
[eval] step 10650 | train 1.1826 | val 4.8995 | elapsed 99.4s
step 10700/30000 | loss 2.0663
step 10750/30000 | loss 1.9802
```

```
step 10800/30000 | loss 1.8926
[eval] step 10800 | train 1.1769 | val 4.8921 | elapsed 100.5s
step 10850/30000 | loss 1.8822
step 10900/30000 | loss 1.9101
step 10950/30000 | loss 1.8996
[eval] step 10950 | train 1.1428 | val 4.9326 | elapsed 101.6s
step 11000/30000 | loss 1.8710
step 11050/30000 | loss 1.9181
step 11100/30000 | loss 1.9081
[eval] step 11100 | train 1.1411 | val 4.9519 | elapsed 102.7s
step 11150/30000 | loss 1.8927
step 11200/30000 | loss 1.8687
step 11250/30000 | loss 1.8761
[eval] step 11250 | train 1.1116 | val 4.9776 | elapsed 103.8s
step 11300/30000 | loss 1.8932
step 11350/30000 | loss 1.8611
step 11400/30000 | loss 1.9015
[eval] step 11400 | train 1.1138 | val 4.9741 | elapsed 105.4s
step 11450/30000 | loss 1.8212
step 11500/30000 | loss 1.8600
step 11550/30000 | loss 1.7954
[eval] step 11550 | train 1.0782 | val 5.0066 | elapsed 106.6s
step 11600/30000 | loss 1.8681
step 11650/30000 | loss 1.9336
step 11700/30000 | loss 1.9313
[eval] step 11700 | train 1.0723 | val 5.0029 | elapsed 108.5s
step 11750/30000 | loss 1.8884
step 11800/30000 | loss 1.7241
step 11850/30000 | loss 1.9312
[eval] step 11850 | train 1.0510 | val 5.0197 | elapsed 109.9s
step 11900/30000 | loss 1.8660
step 11950/30000 | loss 1.7840
step 12000/30000 | loss 1.8877
[eval] step 12000 | train 1.0358 | val 5.0051 | elapsed 111.3s
step 12050/30000 | loss 1.7932
step 12100/30000 | loss 1.7869
step 12150/30000 | loss 1.8069
[eval] step 12150 | train 1.0298 | val 5.0497 | elapsed 112.9s
step 12200/30000 | loss 1.7302
step 12250/30000 | loss 1.8134
step 12300/30000 | loss 1.7804
```

```
[eval] step 12300 | train 1.0156 | val 5.0689 | elapsed 114.6s
step 12350/30000 | loss 1.8878
step 12400/30000 | loss 1.8317
step 12450/30000 | loss 1.7455
[eval] step 12450 | train 1.0038 | val 5.0801 | elapsed 116.0s
step 12500/30000 | loss 1.8265
step 12550/30000 | loss 1.8117
step 12600/30000 | loss 1.7722
[eval] step 12600 | train 0.9848 | val 5.0705 | elapsed 117.6s
step 12650/30000 | loss 1.8070
step 12700/30000 | loss 1.8213
step 12750/30000 | loss 1.7252
[eval] step 12750 | train 0.9791 | val 5.0964 | elapsed 119.4s
step 12800/30000 | loss 1.6970
step 12850/30000 | loss 1.8111
step 12900/30000 | loss 1.7054
[eval] step 12900 | train 0.9606 | val 5.1190 | elapsed 121.2s
step 12950/30000 | loss 1.7747
step 13000/30000 | loss 1.7853
step 13050/30000 | loss 1.7126
[eval] step 13050 | train 0.9547 | val 5.1149 | elapsed 122.4s
step 13100/30000 | loss 1.7223
step 13150/30000 | loss 1.7953
step 13200/30000 | loss 1.6615
[eval] step 13200 | train 0.9411 | val 5.1698 | elapsed 123.9s
step 13250/30000 | loss 1.7482
step 13300/30000 | loss 1.7942
step 13350/30000 | loss 1.8236
[eval] step 13350 | train 0.9301 | val 5.1091 | elapsed 125.7s
step 13400/30000 | loss 1.6776
step 13450/30000 | loss 1.6950
step 13500/30000 | loss 1.7130
[eval] step 13500 | train 0.9164 | val 5.2211 | elapsed 127.0s
step 13550/30000 | loss 1.7538
step 13600/30000 | loss 1.7613
step 13650/30000 | loss 1.7089
[eval] step 13650 | train 0.9324 | val 5.1435 | elapsed 128.4s
step 13700/30000 | loss 1.6476
step 13750/30000 | loss 1.7203
step 13800/30000 | loss 1.7163
[eval] step 13800 | train 0.8915 | val 5.1642 | elapsed 130.2s
```

```
step 13850/30000 | loss 1.7881
step 13900/30000 | loss 1.7028
step 13950/30000 | loss 1.7099
[eval] step 13950 | train 0.8964 | val 5.1886 | elapsed 131.9s
step 14000/30000 | loss 1.7220
step 14050/30000 | loss 1.6914
step 14100/30000 | loss 1.7773
[eval] step 14100 | train 0.8611 | val 5.2366 | elapsed 133.7s
step 14150/30000 | loss 1.7144
step 14200/30000 | loss 1.6918
step 14250/30000 | loss 1.7115
[eval] step 14250 | train 0.8450 | val 5.1908 | elapsed 135.3s
step 14300/30000 | loss 1.6684
step 14350/30000 | loss 1.6915
step 14400/30000 | loss 1.6740
[eval] step 14400 | train 0.8533 | val 5.2522 | elapsed 137.0s
step 14450/30000 | loss 1.6631
step 14500/30000 | loss 1.7122
step 14550/30000 | loss 1.6360
[eval] step 14550 | train 0.8391 | val 5.2324 | elapsed 138.3s
step 14600/30000 | loss 1.6066
step 14650/30000 | loss 1.6165
step 14700/30000 | loss 1.6932
[eval] step 14700 | train 0.8237 | val 5.2560 | elapsed 140.0s
step 14750/30000 | loss 1.6435
step 14800/30000 | loss 1.6328
step 14850/30000 | loss 1.6197
[eval] step 14850 | train 0.8286 | val 5.3020 | elapsed 141.4s
step 14900/30000 | loss 1.5761
step 14950/30000 | loss 1.6014
step 15000/30000 | loss 1.5930
[eval] step 15000 | train 0.8332 | val 5.2445 | elapsed 142.7s
step 15050/30000 | loss 1.7073
step 15100/30000 | loss 1.6423
step 15150/30000 | loss 1.6616
[eval] step 15150 | train 0.8002 | val 5.2511 | elapsed 144.3s
step 15200/30000 | loss 1.6129
step 15250/30000 | loss 1.6260
step 15300/30000 | loss 1.6228
[eval] step 15300 | train 0.8110 | val 5.2576 | elapsed 145.7s
step 15350/30000 | loss 1.5860
```

```
step 15400/30000 | loss 1.5628
step 15450/30000 | loss 1.6504
[eval] step 15450 | train 0.7902 | val 5.2811 | elapsed 147.3s
step 15500/30000 | loss 1.6484
step 15550/30000 | loss 1.6347
step 15600/30000 | loss 1.6545
[eval] step 15600 | train 0.7906 | val 5.3043 | elapsed 148.7s
step 15650/30000 | loss 1.6868
step 15700/30000 | loss 1.6091
step 15750/30000 | loss 1.6219
[eval] step 15750 | train 0.7766 | val 5.2969 | elapsed 149.9s
step 15800/30000 | loss 1.6375
step 15850/30000 | loss 1.6159
step 15900/30000 | loss 1.5533
[eval] step 15900 | train 0.7678 | val 5.2819 | elapsed 151.2s
step 15950/30000 | loss 1.5808
step 16000/30000 | loss 1.5791
step 16050/30000 | loss 1.6120
[eval] step 16050 | train 0.7634 | val 5.3244 | elapsed 152.5s
step 16100/30000 | loss 1.6024
step 16150/30000 | loss 1.4865
step 16200/30000 | loss 1.5344
[eval] step 16200 | train 0.7671 | val 5.3457 | elapsed 154.5s
step 16250/30000 | loss 1.5569
step 16300/30000 | loss 1.5435
step 16350/30000 | loss 1.6412
[eval] step 16350 | train 0.7402 | val 5.3701 | elapsed 155.4s
step 16400/30000 | loss 1.5026
step 16450/30000 | loss 1.6708
step 16500/30000 | loss 1.5363
[eval] step 16500 | train 0.7280 | val 5.3047 | elapsed 157.0s
step 16550/30000 | loss 1.5048
step 16600/30000 | loss 1.5113
step 16650/30000 | loss 1.5760
[eval] step 16650 | train 0.7486 | val 5.3912 | elapsed 158.0s
step 16700/30000 | loss 1.6540
step 16750/30000 | loss 1.5981
step 16800/30000 | loss 1.5147
[eval] step 16800 | train 0.7122 | val 5.3325 | elapsed 159.8s
step 16850/30000 | loss 1.5160
step 16900/30000 | loss 1.6286
```

```
step 16950/30000 | loss 1.6039
[eval] step 16950 | train 0.7177 | val 5.3387 | elapsed 161.7s
step 17000/30000 | loss 1.5660
step 17050/30000 | loss 1.5723
step 17100/30000 | loss 1.6749
[eval] step 17100 | train 0.7293 | val 5.3918 | elapsed 163.5s
step 17150/30000 | loss 1.5182
step 17200/30000 | loss 1.5941
step 17250/30000 | loss 1.6182
[eval] step 17250 | train 0.6963 | val 5.4232 | elapsed 164.6s
step 17300/30000 | loss 1.5569
step 17350/30000 | loss 1.5518
step 17400/30000 | loss 1.4380
[eval] step 17400 | train 0.7047 | val 5.4452 | elapsed 166.0s
step 17450/30000 | loss 1.5991
step 17500/30000 | loss 1.5502
step 17550/30000 | loss 1.5323
[eval] step 17550 | train 0.7012 | val 5.4548 | elapsed 167.1s
step 17600/30000 | loss 1.5629
step 17650/30000 | loss 1.5919
step 17700/30000 | loss 1.5608
[eval] step 17700 | train 0.6906 | val 5.4307 | elapsed 168.2s
step 17750/30000 | loss 1.5361
step 17800/30000 | loss 1.4985
step 17850/30000 | loss 1.6182
[eval] step 17850 | train 0.6836 | val 5.4655 | elapsed 169.7s
step 17900/30000 | loss 1.5452
step 17950/30000 | loss 1.5151
step 18000/30000 | loss 1.4414
[eval] step 18000 | train 0.6740 | val 5.3910 | elapsed 170.9s
step 18050/30000 | loss 1.5330
step 18100/30000 | loss 1.4637
step 18150/30000 | loss 1.5045
[eval] step 18150 | train 0.6807 | val 5.4390 | elapsed 172.0s
step 18200/30000 | loss 1.4413
step 18250/30000 | loss 1.5681
step 18300/30000 | loss 1.5082
[eval] step 18300 | train 0.6596 | val 5.4835 | elapsed 173.0s
step 18350/30000 | loss 1.4448
step 18400/30000 | loss 1.5083
step 18450/30000 | loss 1.4898
```

```
[eval] step 18450 | train 0.6512 | val 5.3729 | elapsed 174.1s
step 18500/30000 | loss 1.5307
step 18550/30000 | loss 1.4975
step 18600/30000 | loss 1.5729
[eval] step 18600 | train 0.6486 | val 5.4898 | elapsed 175.3s
step 18650/30000 | loss 1.5618
step 18700/30000 | loss 1.4921
step 18750/30000 | loss 1.5342
[eval] step 18750 | train 0.6569 | val 5.4533 | elapsed 176.4s
step 18800/30000 | loss 1.5090
step 18850/30000 | loss 1.5271
step 18900/30000 | loss 1.5050
[eval] step 18900 | train 0.6562 | val 5.4812 | elapsed 177.7s
step 18950/30000 | loss 1.4731
step 19000/30000 | loss 1.4825
step 19050/30000 | loss 1.4535
[eval] step 19050 | train 0.6387 | val 5.4801 | elapsed 178.9s
step 19100/30000 | loss 1.4739
step 19150/30000 | loss 1.5130
step 19200/30000 | loss 1.4798
[eval] step 19200 | train 0.6442 | val 5.4961 | elapsed 180.1s
step 19250/30000 | loss 1.4777
step 19300/30000 | loss 1.4873
step 19350/30000 | loss 1.5012
[eval] step 19350 | train 0.6229 | val 5.4581 | elapsed 181.3s
step 19400/30000 | loss 1.4143
step 19450/30000 | loss 1.5377
step 19500/30000 | loss 1.4590
[eval] step 19500 | train 0.6353 | val 5.4847 | elapsed 182.4s
step 19550/30000 | loss 1.4403
step 19600/30000 | loss 1.4565
step 19650/30000 | loss 1.4962
[eval] step 19650 | train 0.6226 | val 5.5167 | elapsed 183.5s
step 19700/30000 | loss 1.5235
step 19750/30000 | loss 1.4734
step 19800/30000 | loss 1.4659
[eval] step 19800 | train 0.6143 | val 5.5287 | elapsed 184.5s
step 19850/30000 | loss 1.4197
step 19900/30000 | loss 1.5241
step 19950/30000 | loss 1.5668
[eval] step 19950 | train 0.5938 | val 5.5216 | elapsed 185.5s
```

```
step 20000/30000 | loss 1.4991
step 20050/30000 | loss 1.5567
step 20100/30000 | loss 1.4808
[eval] step 20100 | train 0.5876 | val 5.5774 | elapsed 186.6s
step 20150/30000 | loss 1.4752
step 20200/30000 | loss 1.4388
step 20250/30000 | loss 1.4337
[eval] step 20250 | train 0.5980 | val 5.5047 | elapsed 187.7s
step 20300/30000 | loss 1.3315
step 20350/30000 | loss 1.4964
step 20400/30000 | loss 1.4635
[eval] step 20400 | train 0.6009 | val 5.5535 | elapsed 188.7s
step 20450/30000 | loss 1.5029
step 20500/30000 | loss 1.4891
step 20550/30000 | loss 1.4770
[eval] step 20550 | train 0.5911 | val 5.5436 | elapsed 189.8s
step 20600/30000 | loss 1.4331
step 20650/30000 | loss 1.4653
step 20700/30000 | loss 1.5208
[eval] step 20700 | train 0.5970 | val 5.6003 | elapsed 191.6s
step 20750/30000 | loss 1.4453
step 20800/30000 | loss 1.4726
step 20850/30000 | loss 1.4377
[eval] step 20850 | train 0.5886 | val 5.5586 | elapsed 193.3s
step 20900/30000 | loss 1.4528
step 20950/30000 | loss 1.4205
step 21000/30000 | loss 1.4198
[eval] step 21000 | train 0.5875 | val 5.6096 | elapsed 194.5s
step 21050/30000 | loss 1.4174
step 21100/30000 | loss 1.4863
step 21150/30000 | loss 1.4707
[eval] step 21150 | train 0.5877 | val 5.6051 | elapsed 195.8s
step 21200/30000 | loss 1.3859
step 21250/30000 | loss 1.4585
step 21300/30000 | loss 1.4751
[eval] step 21300 | train 0.5674 | val 5.6113 | elapsed 197.1s
step 21350/30000 | loss 1.3780
step 21400/30000 | loss 1.4174
step 21450/30000 | loss 1.5111
[eval] step 21450 | train 0.5803 | val 5.5599 | elapsed 198.6s
step 21500/30000 | loss 1.4002
```

```
step 21550/30000 | loss 1.4567
step 21600/30000 | loss 1.3865
[eval] step 21600 | train 0.5816 | val 5.6186 | elapsed 199.9s
step 21650/30000 | loss 1.3458
step 21700/30000 | loss 1.4686
step 21750/30000 | loss 1.4553
[eval] step 21750 | train 0.5704 | val 5.5978 | elapsed 201.7s
step 21800/30000 | loss 1.4624
step 21850/30000 | loss 1.4738
step 21900/30000 | loss 1.3982
[eval] step 21900 | train 0.5689 | val 5.5665 | elapsed 203.5s
step 21950/30000 | loss 1.4417
step 22000/30000 | loss 1.4583
step 22050/30000 | loss 1.3916
[eval] step 22050 | train 0.5464 | val 5.5572 | elapsed 205.4s
step 22100/30000 | loss 1.4462
step 22150/30000 | loss 1.4579
step 22200/30000 | loss 1.4803
[eval] step 22200 | train 0.5563 | val 5.6693 | elapsed 207.3s
step 22250/30000 | loss 1.4114
step 22300/30000 | loss 1.3493
step 22350/30000 | loss 1.3690
[eval] step 22350 | train 0.5498 | val 5.6209 | elapsed 208.7s
step 22400/30000 | loss 1.4482
step 22450/30000 | loss 1.4715
step 22500/30000 | loss 1.4810
[eval] step 22500 | train 0.5603 | val 5.6318 | elapsed 210.2s
step 22550/30000 | loss 1.4508
step 22600/30000 | loss 1.3946
step 22650/30000 | loss 1.3316
[eval] step 22650 | train 0.5478 | val 5.6417 | elapsed 211.6s
step 22700/30000 | loss 1.3807
step 22750/30000 | loss 1.4242
step 22800/30000 | loss 1.3588
[eval] step 22800 | train 0.5383 | val 5.6236 | elapsed 212.6s
step 22850/30000 | loss 1.4767
step 22900/30000 | loss 1.4079
step 22950/30000 | loss 1.3339
[eval] step 22950 | train 0.5441 | val 5.6539 | elapsed 213.6s
step 23000/30000 | loss 1.4226
step 23050/30000 | loss 1.3422
```

```
step 23100/30000 | loss 1.3727
[eval] step 23100 | train 0.5537 | val 5.6772 | elapsed 215.4s
step 23150/30000 | loss 1.4444
step 23200/30000 | loss 1.3335
step 23250/30000 | loss 1.4206
[eval] step 23250 | train 0.5311 | val 5.6453 | elapsed 216.8s
step 23300/30000 | loss 1.4169
step 23350/30000 | loss 1.3493
step 23400/30000 | loss 1.3650
[eval] step 23400 | train 0.5247 | val 5.6524 | elapsed 218.5s
step 23450/30000 | loss 1.3807
step 23500/30000 | loss 1.3010
step 23550/30000 | loss 1.4739
[eval] step 23550 | train 0.5241 | val 5.6821 | elapsed 219.7s
step 23600/30000 | loss 1.4054
step 23650/30000 | loss 1.4571
step 23700/30000 | loss 1.3866
[eval] step 23700 | train 0.5308 | val 5.6587 | elapsed 221.1s
step 23750/30000 | loss 1.3512
step 23800/30000 | loss 1.3257
step 23850/30000 | loss 1.4393
[eval] step 23850 | train 0.5229 | val 5.6887 | elapsed 222.6s
step 23900/30000 | loss 1.3086
step 23950/30000 | loss 1.4005
step 24000/30000 | loss 1.4097
[eval] step 24000 | train 0.5296 | val 5.6602 | elapsed 224.4s
step 24050/30000 | loss 1.4247
step 24100/30000 | loss 1.4225
step 24150/30000 | loss 1.3675
[eval] step 24150 | train 0.5211 | val 5.7093 | elapsed 226.2s
step 24200/30000 | loss 1.3606
step 24250/30000 | loss 1.3493
step 24300/30000 | loss 1.4432
[eval] step 24300 | train 0.5190 | val 5.7142 | elapsed 227.8s
step 24350/30000 | loss 1.3157
step 24400/30000 | loss 1.3413
step 24450/30000 | loss 1.3524
[eval] step 24450 | train 0.5131 | val 5.7099 | elapsed 229.8s
step 24500/30000 | loss 1.3897
step 24550/30000 | loss 1.3136
step 24600/30000 | loss 1.2988
```

```
[eval] step 24600 | train 0.5140 | val 5.6941 | elapsed 231.6s
step 24650/30000 | loss 1.4224
step 24700/30000 | loss 1.3477
step 24750/30000 | loss 1.4449
[eval] step 24750 | train 0.5059 | val 5.7743 | elapsed 233.3s
step 24800/30000 | loss 1.3249
step 24850/30000 | loss 1.3687
step 24900/30000 | loss 1.3966
[eval] step 24900 | train 0.4989 | val 5.7430 | elapsed 234.9s
step 24950/30000 | loss 1.3544
step 25000/30000 | loss 1.3213
step 25050/30000 | loss 1.4049
[eval] step 25050 | train 0.5136 | val 5.7199 | elapsed 236.4s
step 25100/30000 | loss 1.3754
step 25150/30000 | loss 1.3365
step 25200/30000 | loss 1.4060
[eval] step 25200 | train 0.4973 | val 5.7259 | elapsed 238.2s
step 25250/30000 | loss 1.3685
step 25300/30000 | loss 1.3812
step 25350/30000 | loss 1.3487
[eval] step 25350 | train 0.5090 | val 5.7298 | elapsed 239.9s
step 25400/30000 | loss 1.3787
step 25450/30000 | loss 1.3673
step 25500/30000 | loss 1.3407
[eval] step 25500 | train 0.4941 | val 5.7551 | elapsed 241.7s
step 25550/30000 | loss 1.3243
step 25600/30000 | loss 1.3165
step 25650/30000 | loss 1.4039
[eval] step 25650 | train 0.4794 | val 5.7136 | elapsed 243.3s
step 25700/30000 | loss 1.3603
step 25750/30000 | loss 1.4126
step 25800/30000 | loss 1.3183
[eval] step 25800 | train 0.4801 | val 5.7924 | elapsed 244.9s
step 25850/30000 | loss 1.3913
step 25900/30000 | loss 1.2912
step 25950/30000 | loss 1.3308
[eval] step 25950 | train 0.4928 | val 5.7669 | elapsed 246.1s
step 26000/30000 | loss 1.4224
step 26050/30000 | loss 1.3603
step 26100/30000 | loss 1.3828
[eval] step 26100 | train 0.4928 | val 5.7261 | elapsed 247.3s
```

```
step 26150/30000 | loss 1.3591
step 26200/30000 | loss 1.3337
step 26250/30000 | loss 1.3309
[eval] step 26250 | train 0.4871 | val 5.7754 | elapsed 248.3s
step 26300/30000 | loss 1.3292
step 26350/30000 | loss 1.3129
step 26400/30000 | loss 1.3389
[eval] step 26400 | train 0.4780 | val 5.7735 | elapsed 249.5s
step 26450/30000 | loss 1.4632
step 26500/30000 | loss 1.3763
step 26550/30000 | loss 1.3460
[eval] step 26550 | train 0.4863 | val 5.8152 | elapsed 250.4s
step 26600/30000 | loss 1.3399
step 26650/30000 | loss 1.3161
step 26700/30000 | loss 1.2472
[eval] step 26700 | train 0.4749 | val 5.7604 | elapsed 251.5s
step 26750/30000 | loss 1.2724
step 26800/30000 | loss 1.2989
step 26850/30000 | loss 1.4095
[eval] step 26850 | train 0.4766 | val 5.8193 | elapsed 252.5s
step 26900/30000 | loss 1.3701
step 26950/30000 | loss 1.2619
step 27000/30000 | loss 1.2741
[eval] step 27000 | train 0.4684 | val 5.7977 | elapsed 253.9s
step 27050/30000 | loss 1.3730
step 27100/30000 | loss 1.3338
step 27150/30000 | loss 1.3685
[eval] step 27150 | train 0.4709 | val 5.8110 | elapsed 254.9s
step 27200/30000 | loss 1.2825
step 27250/30000 | loss 1.2429
step 27300/30000 | loss 1.2888
[eval] step 27300 | train 0.4777 | val 5.7660 | elapsed 255.8s
step 27350/30000 | loss 1.3479
step 27400/30000 | loss 1.3022
step 27450/30000 | loss 1.3334
[eval] step 27450 | train 0.4606 | val 5.8375 | elapsed 256.8s
step 27500/30000 | loss 1.3312
step 27550/30000 | loss 1.3381
step 27600/30000 | loss 1.3498
[eval] step 27600 | train 0.4651 | val 5.7970 | elapsed 257.8s
step 27650/30000 | loss 1.2976
```

```
step 27700/30000 | loss 1.3225
step 27750/30000 | loss 1.3190
[eval] step 27750 | train 0.4726 | val 5.8112 | elapsed 258.7s
step 27800/30000 | loss 1.2611
step 27850/30000 | loss 1.4220
step 27900/30000 | loss 1.3350
[eval] step 27900 | train 0.4655 | val 5.8770 | elapsed 259.6s
step 27950/30000 | loss 1.3513
step 28000/30000 | loss 1.3188
step 28050/30000 | loss 1.2523
[eval] step 28050 | train 0.4709 | val 5.7863 | elapsed 260.6s
step 28100/30000 | loss 1.3645
step 28150/30000 | loss 1.3147
step 28200/30000 | loss 1.3118
[eval] step 28200 | train 0.4564 | val 5.8000 | elapsed 261.9s
step 28250/30000 | loss 1.3324
step 28300/30000 | loss 1.3697
step 28350/30000 | loss 1.2985
[eval] step 28350 | train 0.4643 | val 5.7764 | elapsed 263.1s
step 28400/30000 | loss 1.2172
step 28450/30000 | loss 1.3695
step 28500/30000 | loss 1.2994
[eval] step 28500 | train 0.4560 | val 5.7976 | elapsed 264.8s
step 28550/30000 | loss 1.3259
step 28600/30000 | loss 1.3173
step 28650/30000 | loss 1.3296
[eval] step 28650 | train 0.4525 | val 5.8761 | elapsed 266.5s
step 28700/30000 | loss 1.3262
step 28750/30000 | loss 1.2786
step 28800/30000 | loss 1.3419
[eval] step 28800 | train 0.4487 | val 5.8069 | elapsed 267.7s
step 28850/30000 | loss 1.3158
step 28900/30000 | loss 1.3153
step 28950/30000 | loss 1.3199
[eval] step 28950 | train 0.4548 | val 5.8394 | elapsed 268.8s
step 29000/30000 | loss 1.1834
step 29050/30000 | loss 1.1796
step 29100/30000 | loss 1.3236
[eval] step 29100 | train 0.4468 | val 5.8288 | elapsed 270.0s
step 29150/30000 | loss 1.2783
step 29200/30000 | loss 1.3486
```

```
step 29250/30000 | loss 1.3396
[eval] step 29250 | train 0.4412 | val 5.8404 | elapsed 271.4s
step 29300/30000 | loss 1.3803
step 29350/30000 | loss 1.2555
step 29400/30000 | loss 1.2884
[eval] step 29400 | train 0.4365 | val 5.8522 | elapsed 272.6s
step 29450/30000 | loss 1.2386
step 29500/30000 | loss 1.2968
step 29550/30000 | loss 1.2933
[eval] step 29550 | train 0.4436 | val 5.8616 | elapsed 273.7s
step 29600/30000 | loss 1.3792
step 29650/30000 | loss 1.2989
step 29700/30000 | loss 1.2544
[eval] step 29700 | train 0.4441 | val 5.8943 | elapsed 274.7s
step 29750/30000 | loss 1.3634
step 29800/30000 | loss 1.2329
step 29850/30000 | loss 1.2918
[eval] step 29850 | train 0.4413 | val 5.8739 | elapsed 275.8s
step 29900/30000 | loss 1.2611
step 29950/30000 | loss 1.2767
step 30000/30000 | loss 1.2926
[eval] step 30000 | train 0.4412 | val 5.8588 | elapsed 276.9s
```

Generate text

In [43]:

```python
# create an initial prompt and encode it using the tokenizer to get the starting token IDs for generation.

prompt = "ROMEO:"

# idx is the tensor of token IDs corresponding to the prompt, which will be used as the initial input to the model
idx = torch.tensor([tokenizer.encode(prompt)], dtype=torch.long, device=device)

# We call the generate method of the model to produce new token IDs based on the initial prompt.
# The generate method will produce a sequence of token IDs that represent the generated text, which
# we then decode back into a string using the tokenizer's decode method.
# max_new_tokens specifies how many new tokens to generate
# top_k means that at each step of generation, we will only consider the top_k most likely next tokens to sample fr
# which can help improve the quality of the generated text by avoiding low-probability tokens.
# temperature controls the randomness of the sampling process; higher temperature leads to more random sampling,
```

```
# while lower temperature makes the model more deterministic.
out = model.generate(idx, max_new_tokens=250, top_k=50, temperature=0.9)[0].tolist()

# Finally, we decode the generated token IDs back into a string to see the generated text.
print(tokenizer.decode(out))
```

```
ROMEO: the gods glads sit on your hands
And present learn heart the top: you
Have topen it you with you a suitor. Camillo
Put not your proclaimed again: all kinded
Give me my bold till I can show her denvy my
Where you pare not?

FLORIZEL:
I now on that when my
Provost, alsily the gates and sweet friends
Reither be in safe. They are commission.

FLORIZEL:
O Perdita, what have we twain forgot!
O that need I have done access of her
Under should the youth, was the bond
Whichhe's another of the matter.

CAMILLO:
I cannot do it not, but once
But will a dangerous pointestion as I may wear
From childish for yourselves: swears I dispatch in my
```

Inspect next-token probabilities

In [44]:

```
def topk_next_tokens(prompt: str, k: int = 12):
    ids = tokenizer.encode(prompt)
    idx = torch.tensor([ids[-cfg.block_size:]], dtype=torch.long, device=device)
    with torch.no_grad():
        logits, _ = model(idx)
        probs = torch.softmax(logits[0, -1], dim=-1)

    p, tok = torch.topk(probs, k=k)
    rows = []
```

```
    for pi, ti in zip(p.tolist(), tok.tolist()):
        piece = tokenizer.id_to_bytes[ti].decode("utf-8", errors="replace").replace("\n","\\n")
        rows.append((ti, pi, piece))
    return rows

for tid, pi, piece in topk_next_tokens("ROMEO: ", k=12):
    print(f"id={tid:4d} prob={pi:.4f} piece='{piece}'")
```

```
id= 745 prob=0.1569 piece='yet '
id= 415 prob=0.1153 piece='if'
id= 688 prob=0.0970 piece='O, '
id= 323 prob=0.0562 piece='our'
id= 313 prob=0.0363 piece='I '
id=  79 prob=0.0353 piece='O'
id= 720 prob=0.0271 piece='look'
id= 279 prob=0.0266 piece='is '
id= 300 prob=0.0249 piece='for'
id= 610 prob=0.0206 piece='let '
id= 121 prob=0.0199 piece='y'
id= 386 prob=0.0194 piece='now'
```

Common Autoregressive (next-token) foundation models

> **Autoregressive** means the model is trained to predict the next token: $p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$.
> Most modern LLM chat systems are deployed as **decoder-style Transformer** generators (often with MoE, tools, and multimodal input).

Provider	Model (representative)	Open weights?	Encoder/Decoder style	Context window (typical)	Modalities	Notes / distinguishing traits
OpenAI	GPT-5 (plus GPT-5 mini / nano)	No	Decoder-style Transformer (AR)	400K context, 128K max output	Text + vision (image input)	Frontier general model family; API variants for speed/cost

Provider	Model (representative)	Open weights?	Encoder/Decoder style	Context window (typical)	Modalities	Notes / distinguishing traits
Anthropic	Claude Opus 4.6	No	Decoder-style Transformer (AR)	200K standard; 1M context in beta	Text (tool use); product supports agentic workflows	“Extended thinking” controls; long-context focus
Anthropic	Claude Sonnet 4.6	No	Decoder-style Transformer (AR)	200K standard; 1M context in beta	Text (tool use)	Balanced speed/intelligence tier; long-context planning
Google	Gemini 2.0 Flash	No	Autoregressive generative model (Transformer-based)	1M context	Multimodal (text + images etc.)	Built-in tool use positioning; long-context support
DeepSeek (API)	deepseek-chat / deepseek-reasoner (V3.2)	No	Decoder-style Transformer (AR)	128K context	Text	“Chat” vs “Reasoner” endpoints map to V3.2 non-thinking vs thinking mode
DeepSeek (open)	DeepSeek-V3 (MoE)	Yes	Decoder-style Transformer (AR), **MoE**	(commonly 128K in ecosystem)	Text	MoE with large total params / smaller active params; efficiency focus
DeepSeek (open)	DeepSeek-R1	Yes	Decoder-style Transformer (AR)	(varies by hosting; see model card/docs)	Text	Reasoning-oriented family; includes distilled variants released to community
Meta	Llama 4 Scout	Partial (“open-weight” under Meta license)	Decoder-style Transformer (AR), **MoE**	10M context	Text + image input; text output	Very long context emphasis
Meta	Llama 4 Maverick	Partial (“open-weight” under Meta license)	Decoder-style Transformer (AR), **MoE**	1M context	Text + image input; text output	Strong general performance tier

Provider	Model (representative)	Open weights?	Encoder/Decoder style	Context window (typical)	Modalities	Notes / distinguishing traits
Mistral	Mistral Large 3	Yes (Apache 2.0)	Decoder-style Transformer (AR), **MoE**	256K context	Multimodal (per model docs)	Open-weight flagship; large MoE with smaller active params
Alibaba / Qwen	Qwen2.5-1M (family)	Mixed (many open releases)	Decoder-style Transformer (AR)	1M context	Text (plus ecosystem multimodal variants)	Long-context training/extrapolation line
xAI	Grok 4.1 Fast (reasoning / non-reasoning variants)	No	Decoder-style Transformer (AR)	2M context	Text (+ agent tools; visual processing mentioned)	Tool-calling / agentic workflows; long-context RL emphasis

Sources (links)

OpenAI

- https://openai.com/gpt-5/
- https://developers.openai.com/api/docs/models/gpt-5
- https://developers.openai.com/api/docs/models/gpt-5-mini
- https://developers.openai.com/api/docs/models/gpt-5-nano

Anthropic (Claude 4.6)

- https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-6
- https://www.anthropic.com/news/claude-opus-4-6
- https://www.anthropic.com/news/claude-sonnet-4-6

DeepSeek

- https://api-docs.deepseek.com/quick_start/pricing
- https://github.com/deepseek-ai/DeepSeek-V3

- https://huggingface.co/deepseek-ai/DeepSeek-R1

Meta (Llama 4)

- https://ai.meta.com/blog/llama-4-multimodal-intelligence/
- https://www.llama.com/models/llama-4/

Google (Gemini)

- https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash

Mistral

- https://docs.mistral.ai/models/mistral-large-3-25-12
- https://mistral.ai/news/mistral-3

Qwen

- https://qwenlm.github.io/blog/qwen2.5-1m/

xAI (Grok)

- https://x.ai/news/grok-4-1-fast
- https://x.ai/api

Other Transformer architectures: decoder-only vs encoder-only vs encoder–decoder

Architecture	Core idea	Attention pattern	Typical training objective	Strengths	Common uses	Representative model families
Decoder-only (causal LM)	Generate next token from the prefix	**Causal self-attention** (token t	**Autoregressive next-token prediction**	Best for open-ended generation; scales well; natural for chat/code	Chatbots, text generation, code generation, long-form	GPT (OpenAI), Claude (Anthropic), Gemini (Google;

Architecture	Core idea	Attention pattern	Typical training objective	Strengths	Common uses	Representative model families
		attends only to $\leq t$)			writing, agentic tool use	generative), Llama (Meta), Mistral, DeepSeek, Qwen, Grok
Encoder-only (bidirectional encoder)	Build contextual representations using full input context	**Bidirectional self-attention** (each token attends to all tokens)	**Masked language modeling** (or discriminative pretraining)	Strong understanding/representation learning; great for classification/retrieval	Classification, NER, search/ranking, embeddings, feature extraction	BERT, RoBERTa, DeBERTa, ELECTRA, ALBERT, ModernBERT-like families
Encoder–decoder (seq2seq)	Encode input, then decode output conditioned on encoded memory	Encoder: bidirectional self-attn; Decoder: causal self-attn + **cross-attention** to encoder	**Conditional generation** (teacher-forced seq2seq)	Excellent for input→output transforms; controllable generation	Translation, summarization, question answering, rewriting, structured generation	T5 / FLAN-T5, BART, mT5, UL2, MarianMT

Objectives (equations)

Decoder-only (autoregressive) objective

Given tokens $s_{1:T}$: $p_\theta(s_{1:T}) = \prod_{t=1}^{T} p_\theta(s_t \mid s_{<t})$, and the negative log-likelihood loss: $\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(s_t \mid s_{<t})$.

Encoder-only (masked language modeling)

Let M be the set of masked positions. The model predicts only masked tokens:

$\mathcal{L}(\theta) = -\sum_{t \in M} \log p_\theta(s_t \mid s_{\setminus M})$.

Encoder–decoder (seq2seq conditional generation)

Given an input sequence x and output sequence $y_{1:T}$:

$$p_\theta(y_{1:T} \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x),$$

and the training loss:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}, x).$$

Some current llm architectures

- https://sebastianraschka.com/llm-architecture-gallery/

Summary

- **Decoder-only:** best when you want to *generate* continuations.
- **Encoder-only:** best when you want to *understand* or *embed/classify* inputs.
- **Encoder–decoder:** best when you want to *transform* one sequence into another (translation/summarization/etc.).

Use of Generative AI

Portions of this material were developed with the assistance of **generative artificial intelligence tools.
The author reviewed, edited, and validated all content, including explanations, code, and examples, and assumes full responsibility for accuracy and interpretation.